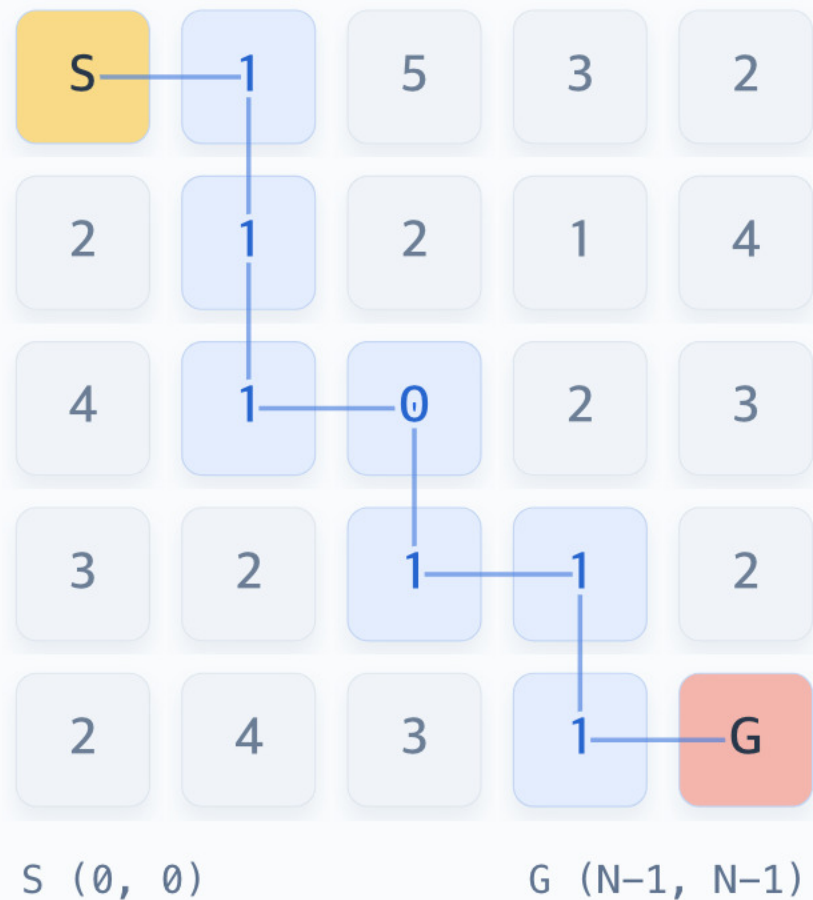


SWEA 1249

보급로

가중치가 있는 격자에서
최소 비용 경로 찾기

— Dijkstra



| 목차

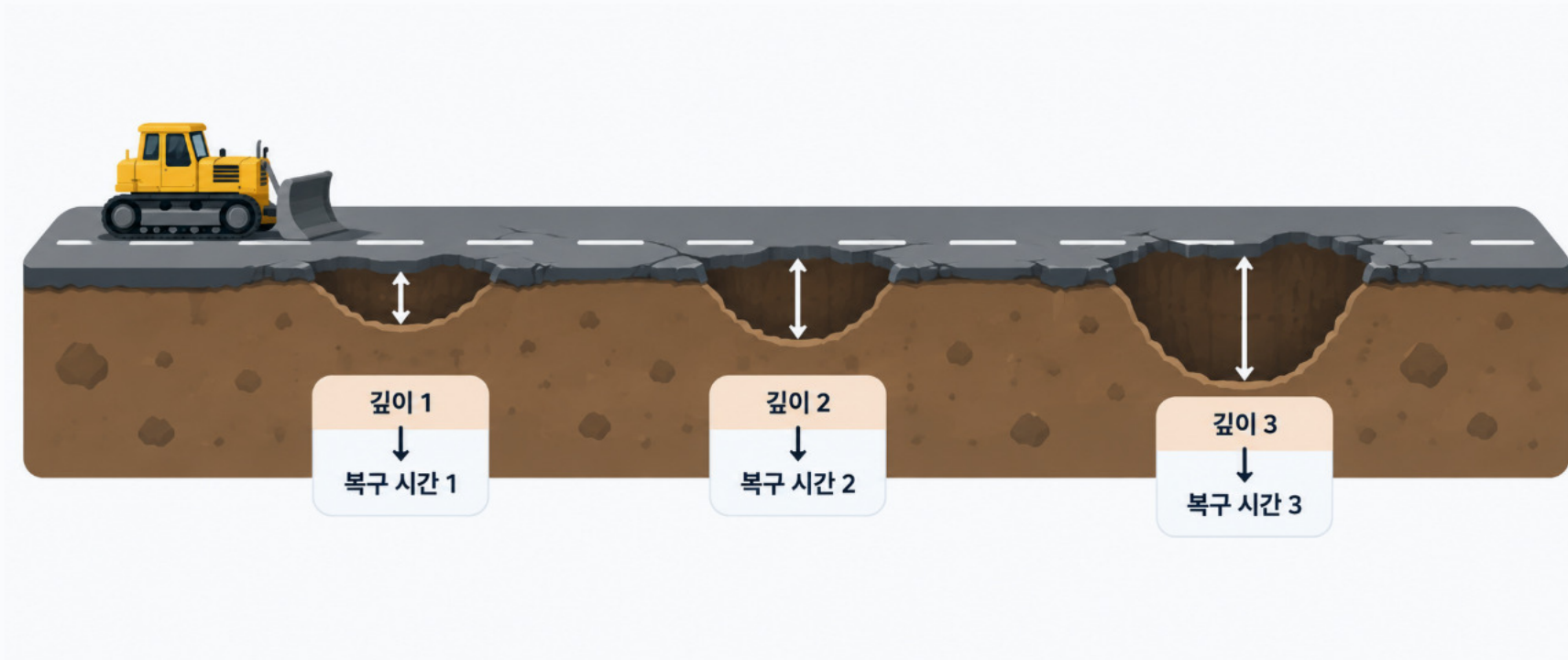
01	문제 이해	이동 규칙과 최소화할 비용	03
02	알고리즘 선택	후보 비교와 다익스트라의 선택 근거	07
03	다익스트라 알고리즘	탐색 순서와 최소 비용 갱신	13
04	보급로에 적용	좌표 기반 코드와 격자 실행	25
05	정리	알고리즘 선택 기준과 핵심 복습	31

01

문제 이해

파손된 도로의 복구 시간을 살펴보고,
출발지에서 도착지까지 무엇을 최소화할지 정리합니다.

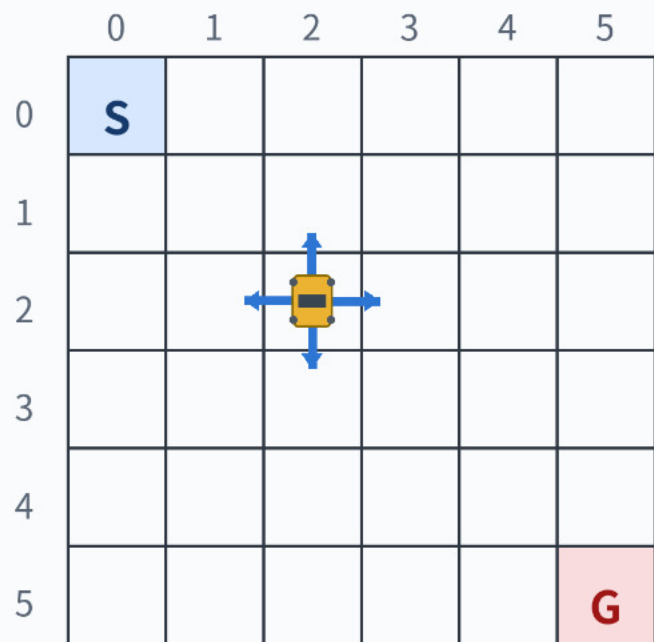
| S에서 G까지, 파손된 도로를 복구해야 한다



출발지 S → 도착지 G

파손 깊이 = 복구 시간

| 한 번에 한 칸, 상하좌우로 이동한다



상 · 하 · 좌 · 우

출발점 S

$(0, 0)$

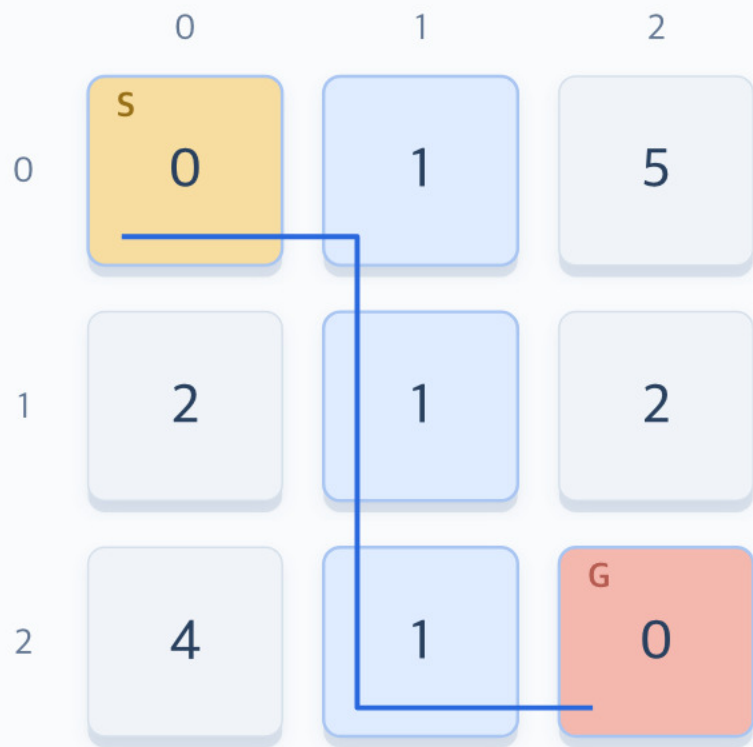
도착점 G

$(N-1, N-1)$

$N \times N$ 격자 · 0-based 인덱스

상 / 하 / 좌 / 우

| 최소화해야 하는 것은 총 복구 시간이다



숫자 = 해당 칸의 복구 시간

이동 시간 자체는 무시한다.

이동 거리 · 이동 횟수

×

복구 시간의 합

✓

칸마다 비용 0~9

모든 비용 ≥ 0

02

알고리즘 선택

경로 탐색 알고리즘의 후보를 비교하고,
BFS의 변례를 통해 다익스트라를 선택하는 이유를 확인합니다.

| 그래프 탐색과 최단 경로의 대표 알고리즘

경로를 찾는 문제도, 목표와 비용 조건에 따라 선택이 달라진다.

알고리즘	주로 검토하는 상황
DFS	도달 가능 여부와 경로 탐색
BFS	이동 비용이 같을 때 최소 이동 횟수
Dijkstra	음수 비용이 없는 단일 출발점 최단 경로
Bellman-Ford	음수 간선도 고려하는 단일 출발점 최단 경로
Floyd-Warshall	모든 정점 쌍 사이의 최단 경로

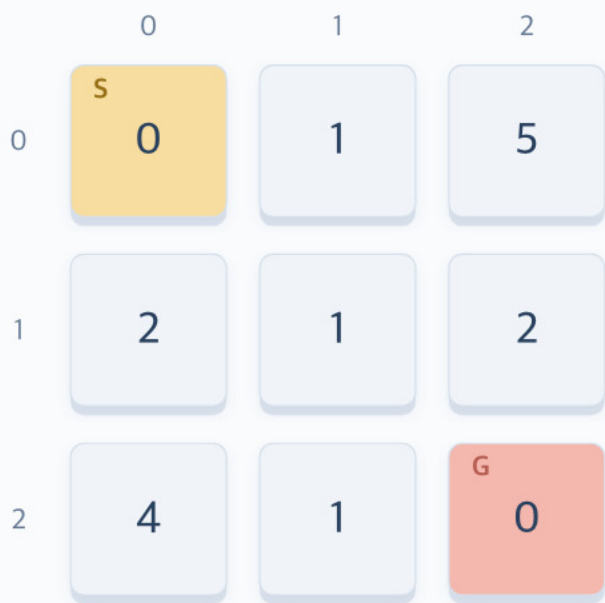
| 그래프 탐색과 최단 경로의 대표 알고리즘

경로를 찾는 문제도, 목표와 비용 조건에 따라 선택이 달라진다.

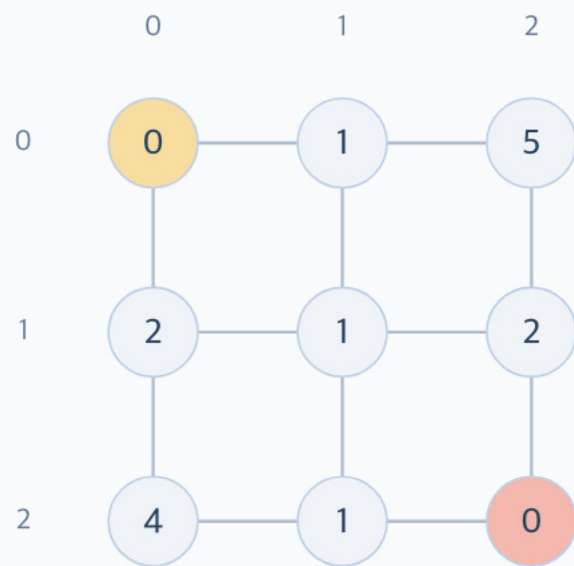
알고리즘	주로 검토하는 상황
DFS	도달 가능 여부와 경로 탐색
BFS	이동 비용이 같을 때 최소 이동 횟수
Dijkstra	음수 비용이 없는 단일 출발점 최단 경로
Bellman-Ford	음수 간선도 고려하는 단일 출발점 최단 경로
Floyd-Warshall	모든 정점 쌍 사이의 최단 경로

격자와 상하좌우 이동을 보면 **BFS**가 떠오른다.

| 격자도 결국 그래프다



숫자 = 해당 칸의 복구 시간



숫자 = 해당 칸의 복구 시간

칸 → 정점 · Vertex

상하좌우 이동 → 간선 · Edge

복구 시간 → 가중치 · Weight

한 출발점에서 누적 복구 시간이 최소인 경로를 찾는 문제다.

| 격자 최단 경로니까, BFS로 풀 수 있을까?



숫자 = 시작점에서의 이동 횟수

격자 + 상하좌우 이동

BFS?

이동 횟수가 적은 칸부터 탐색

0 → 1 → 2 → 3

모든 이동 비용이 같을 때

최소 이동 횟수로 최소 비용도 보장

보급로는 칸마다 복구 시간이 다르다.

| 더 적게 이동하는 길이 더 저렴할까?



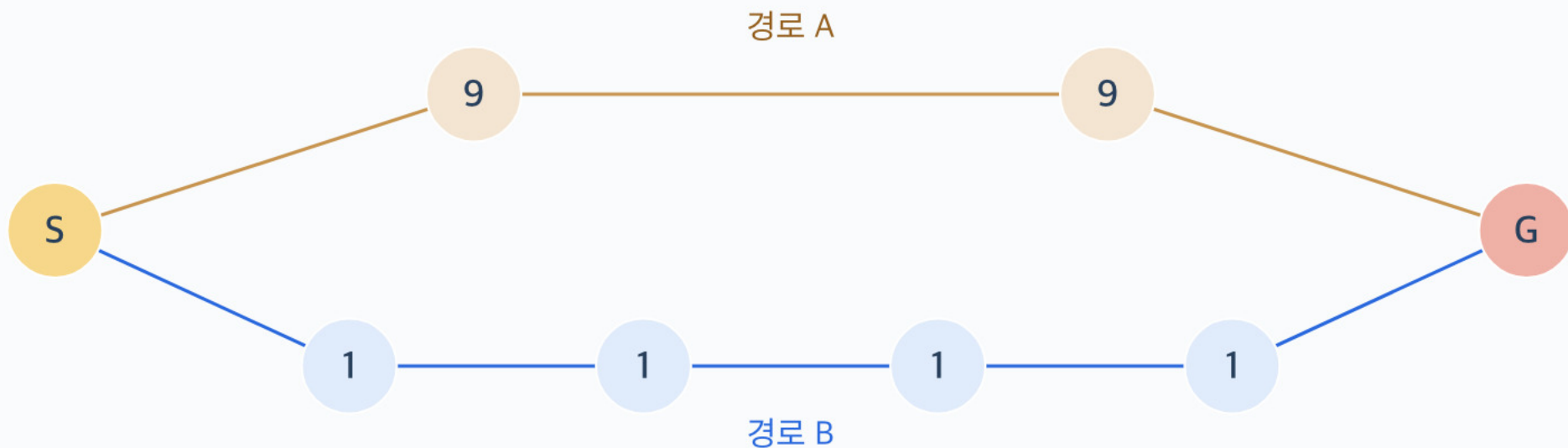
이동 횟수가 최소라고 비용이 최소인 것은 아니다.

| 더 적게 이동하는 길이 더 저렴할까?



이동 횟수가 최소라고 비용이 최소인 것은 아니다.

| 더 적게 이동하는 길이 더 저렴할까?



경로 A

이동 3 | 비용 18

경로 B

이동 5 | 비용 4

이동 횟수가 최소라고 비용이 최소인 것은 아니다.

| 보급로의 조건에 맞는 선택, 다익스트라

출발점 하나

목표 최소 누적 비용

비용 0~9 · 음수 없음

후보

보급로에서의 판단

DFS

기본 DFS만으로는 최소 비용을 보장하지 않음

BFS

이동 비용이 달라 일반 BFS의 보장 조건과 다름

Dijkstra

한 출발점의 최소 비용, 모든 비용이 0 이상

Bellman-Ford

적용 가능하지만, 이 조건에서는 다익스트라가 더 효율적

Floyd-Warshall

모든 정점 쌍의 경로까지 계산할 필요는 없음

누적 비용이 가장 작은 후보부터 탐색한다.

| 보급로의 조건에 맞는 선택, 다익스트라

출발점 하나

목표 최소 누적 비용

비용 0~9 · 음수 없음

후보

보급로에서의 판단

DFS

기본 DFS만으로는 최소 비용을 보장하지 않음

BFS

이동 비용이 달라 일반 BFS의 보장 조건과 다름

Dijkstra

한 출발점의 최소 비용, 모든 비용이 0 이상

Bellman-Ford

적용 가능하지만, 이 조건에서는 다익스트라가 더 효율적

Floyd-Warshall

모든 정점 쌍의 경로까지 계산할 필요는 없음

누적 비용이 가장 작은 후보부터 탐색한다.

| 보급로의 조건에 맞는 선택, 다익스트라

출발점 하나

목표 최소 누적 비용

비용 0~9 · 음수 없음

후보

보급로에서의 판단

DFS

기본 DFS만으로는 최소 비용을 보장하지 않음

BFS

이동 비용이 달라 일반 BFS의 보장 조건과 다름

Dijkstra

한 출발점의 최소 비용, 모든 비용이 0 이상

Bellman-Ford

적용 가능하지만, 이 조건에서는 다익스트라가 더 효율적

Floyd-Warshall

모든 정점 쌍의 경로까지 계산할 필요는 없음

누적 비용이 가장 작은 후보부터 탐색한다.

| 보급로의 조건에 맞는 선택, 다익스트라

출발점 하나

목표 최소 누적 비용

비용 0~9 · 음수 없음

후보

보급로에서의 판단

DFS

기본 DFS만으로는 최소 비용을 보장하지 않음

BFS

이동 비용이 달라 일반 BFS의 보장 조건과 다름

Dijkstra

✓ 한 출발점의 최소 비용, 모든 비용이 0 이상

Bellman-Ford

적용 가능하지만, 이 조건에서는 다익스트라가 더 효율적

Floyd-Warshall

모든 정점 쌍의 경로까지 계산할 필요는 없음

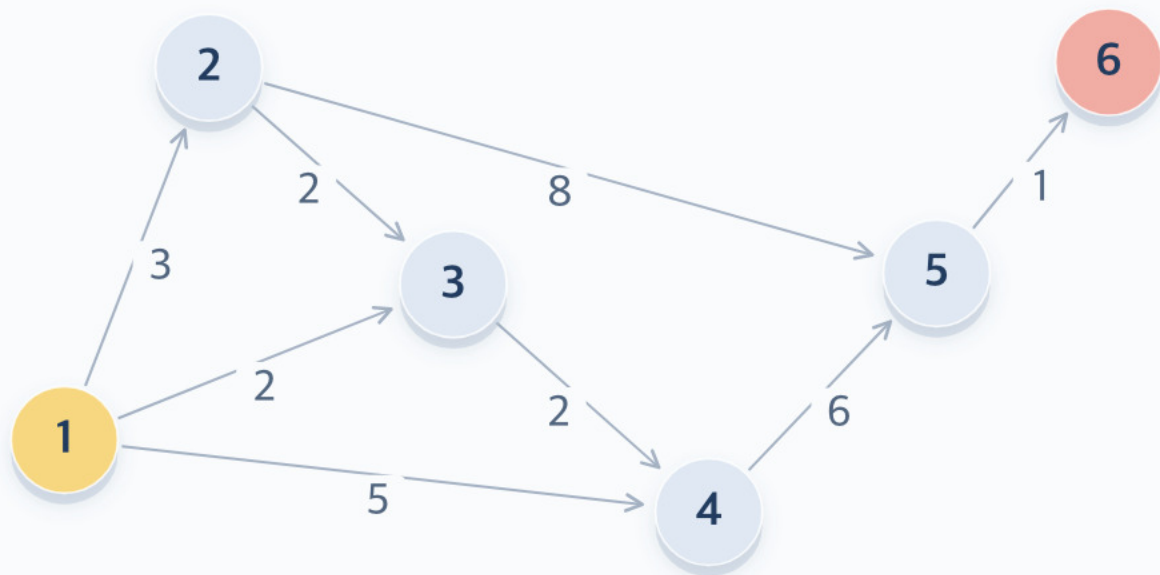
누적 비용이 가장 작은 후보부터 탐색한다.

03

다익스트라 알고리즘

dist와 PQ의 역할, 비용 갱신과 확정의 원리를 배우고,
하나의 그래프에서 전체 실행을 따라갑니다.

| 예제 목표와 최소 비용 후보



출발 1 · 도착 6

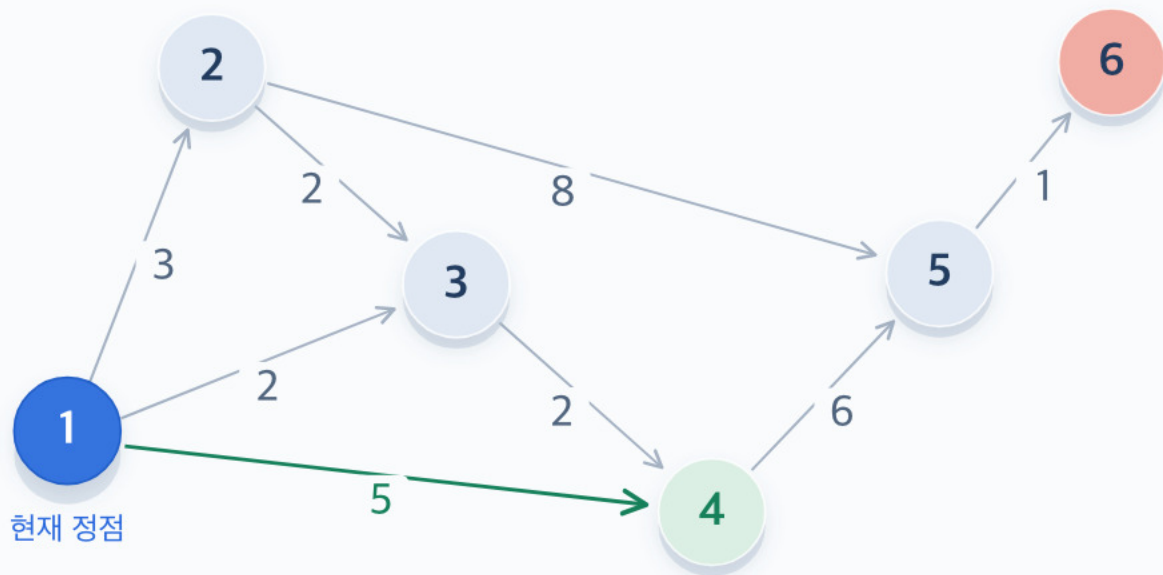
간선 비용의 합이 가장 작은 경로

화살표 방향으로 이동하며,
간선의 숫자만큼 비용을 더한다.

첫 비용은 0에서 시작한다.

목표: 1에서 6까지의 최소 누적 비용

| 예제 목표와 최소 비용 후보



출발 1 · 도착 6

간선 비용의 합이 가장 작은 경로

화살표 방향으로 이동하며,
간선의 숫자만큼 비용을 더한다.

1번의 이웃까지 발견한 누적 비용

정점 3

2

정점 2

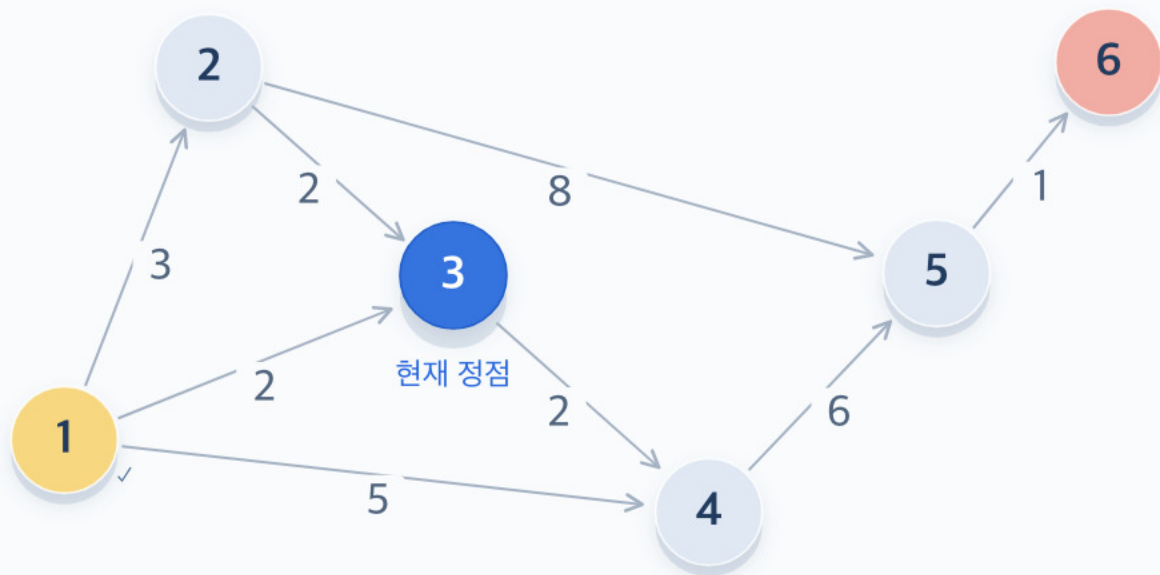
3

정점 4

5

다음에는 어느 후보를 먼저 탐색할까?

| 예제 목표와 최소 비용 후보



출발 1 · 도착 6

간선 비용의 합이 가장 작은 경로

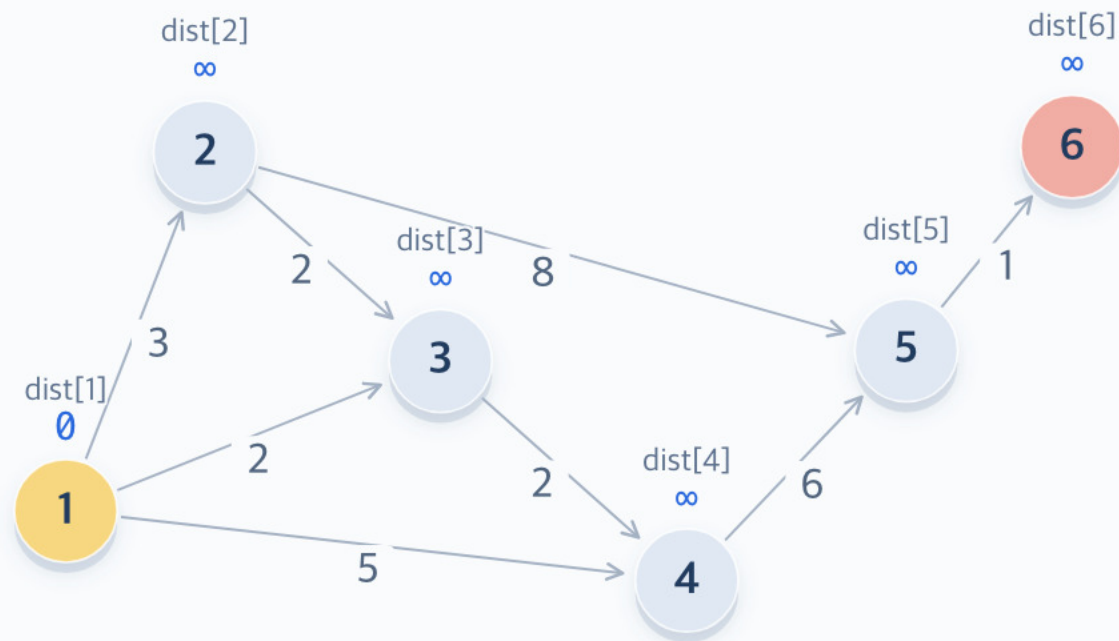
화살표 방향으로 이동하며,
간선의 숫자만큼 비용을 더한다.

1번의 이웃까지 발견한 누적 비용

정점 3	정점 2	정점 4
2	3	5

정점 번호와 관계없이 누적 비용 2인 정점 3을 선택한다.

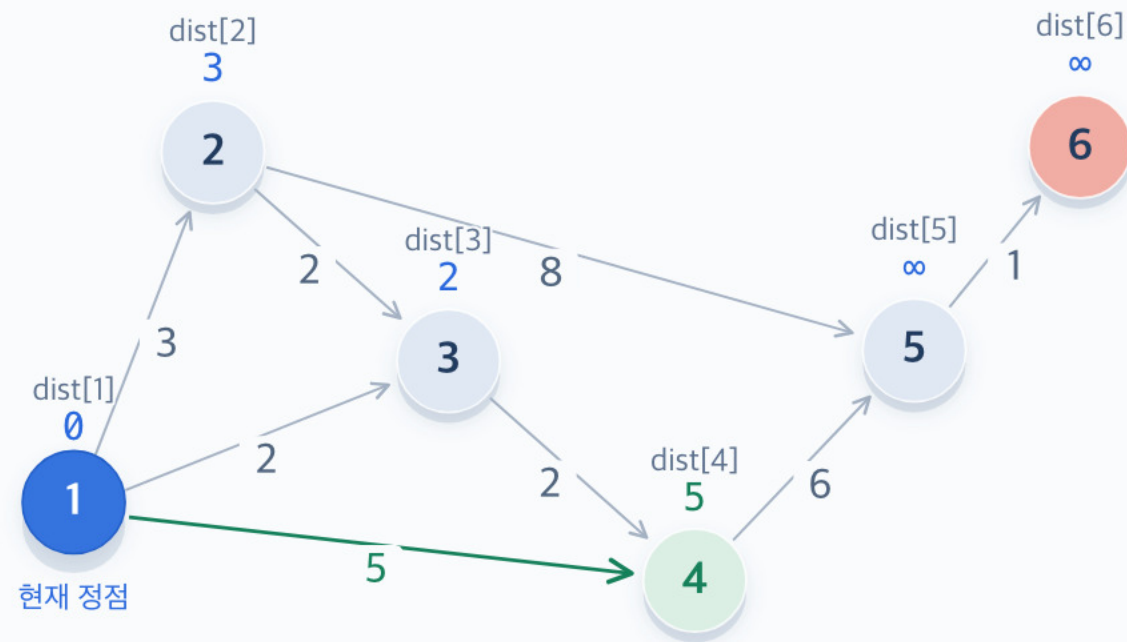
| dist는 현재까지 발견한 최소 비용이다



	1	2	3	4	5	6
dist	0	∞	∞	∞	∞	∞

시작점은 0, 나머지는 ∞ 로 초기화한다. ∞ 는 아직 경로를 찾지 못했다는 뜻이다.

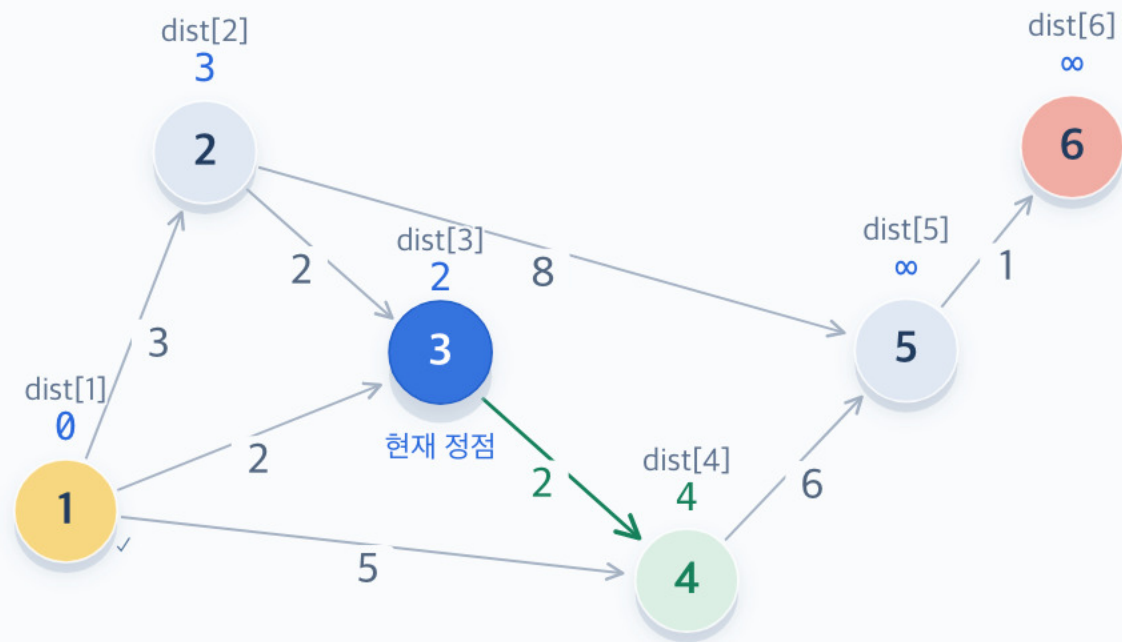
| dist는 현재까지 발견한 최소 비용이다



	1	2	3	4	5	6
dist	0	3	2	5	∞	∞
	확정					

$dist[v]$ 는 현재까지 발견한 최선이다. 확정 전에는 더 작아질 수 있다.

| dist는 현재까지 발견한 최소 비용이다



	1	2	3	4	5	6
dist	0	3	2	4	∞	∞
	확정		확정			

dist[4]는 5에서 4로 갱신된다. PQ에 넣는 시점에는 아직 확정하지 않는다.

| 최소 비용 후보를 먼저 꺼내기 위해 PQ를 쓴다

Priority Queue

정점	누적 비용
3	2 ← 최소
2	3
4	5

후보 = (정점, 기록한 누적 비용)

최소 누적 비용

2 < 3 < 5

poll()  다음 탐색 정점
누적 비용 2

처리할 후보는 PQ에,
최신 최선의 비용은 dist에 기록한다.

| 최소 비용 후보를 먼저 꺼내기 위해 PQ를 쓴다

Priority Queue

정점	누적 비용
2	3 ← 최소
4	5

후보 = (정점, 기록한 누적 비용)

최소 누적 비용

2 < 3 < 5

poll()

3

다음 탐색 정점
누적 비용 2

처리할 후보는 PQ에,
최신 최선의 비용은 dist에 기록한다.

| 후보 하나를 꺼낸 뒤 하는 일

01

최소 후보 꺼내기

```
pq.poll()
```

오래된 기록이면 건너뛰기

02

→ 인접 정점 확인

```
graph[now]
```

03

→ 새 누적 비용 계산

```
currentDistance + cost
```

새 비용 < 기존 dist ?

YES

dist 갱신 + PQ에 추가

NO

기존 값 유지

모든 이웃에 대해 비교한 뒤, 다음 PQ 후보를 꺼낸다.

PQ가 빌 때까지 반복한다.

| 후보 하나를 꺼낸 뒤 하는 일

01

최소 후보 꺼내기

```
pq.poll()
```

오래된 기록이면 건너뛰기

02

인접 정점 확인

```
graph[now]
```

03

새 누적 비용 계산

```
currentDistance + cost
```

새 비용 < 기존 dist ?

YES

dist 갱신 + PQ에 추가

NO

기존 값 유지

모든 이웃에 대해 비교한 뒤, 다음 PQ 후보를 꺼낸다.

PQ가 빌 때까지 반복한다.

| 후보 하나를 꺼낸 뒤 하는 일

01

최소 후보 꺼내기

`pq.poll()`

오래된 기록이면 건너뛰기

02

인접 정점 확인

`graph[now]`

03

새 누적 비용 계산

`currentDistance + cost`

→

→

새 비용 < 기존 dist ?

YES

dist 갱신 + PQ에 추가

NO

기존 값 유지

모든 이웃에 대해 비교한 뒤, 다음 PQ 후보를 꺼낸다.
PQ가 빌 때까지 반복한다.

| 후보 하나를 꺼낸 뒤 하는 일

01

최소 후보 꺼내기

`pq.poll()`

오래된 기록이면 건너뛰기

02

인접 정점 확인

`graph[now]`

03

새 누적 비용 계산

`currentDistance + cost`

→

→

새 비용 < 기존 dist ?

YES

dist 갱신 + PQ에 추가

NO

기존 값 유지

모든 이웃에 대해 비교한 뒤, 다음 PQ 후보를 꺼낸다.
PQ가 빌 때까지 반복한다.

| 후보 하나를 꺼낸 뒤 하는 일

01

최소 후보 꺼내기

`pq.poll()`

오래된 기록이면 건너뛰기

02

인접 정점 확인

`graph[now]`

03

새 누적 비용 계산

`currentDistance + cost`

새 비용 < 기존 dist ?

YES

dist 갱신 + PQ에 추가

NO

기존 값 유지

모든 이웃에 대해 비교한 뒤, 다음 PQ 후보를 꺼낸다.
PQ가 빌 때까지 반복한다.

| 더싼 경로를 발견하면 갱신한다

$\text{dist}[3] = 2$



$$2 + 2 = 4$$

새 비용

4

기존 비용

5

<

$\text{dist}[4]$

5 → 5

갱신 · Relaxation

기존 경로 1→4는 비용 5. 새 경로 1→3→4는?

| 더싼 경로를 발견하면 갱신한다

$\text{dist}[3] = 2$



$$2 + 2 = 4$$

새 비용

4

<

기존 비용

5

$\text{dist}[4]$

5 → 5

갱신 · Relaxation

기존 경로 1→4는 비용 5. 새 경로 1→3→4는?

| 더싼 경로를 발견하면 갱신한다

$\text{dist}[3] = 2$



$$2 + 2 = 4$$

새 비용

4

<

기존 비용

5

$\text{dist}[4]$

5 → 4

갱신 · Relaxation

$\text{dist}[4]$ 를 4로 갱신하고, 새 후보 (4, 4)를 PQ에 추가한다.

| 새로운 경로가 더 비싸면 갱신하지 않는다

$\text{dist}[2] = 3$



$$3 + 2 = 5$$

새 비용

5

기존 비용

2

>

$\text{dist}[3]$

2 = 2

갱신하지 않음

정점 3은 이미 비용 2로 확정했다. 나중에 비교한 $3+2=5$ 로는 갱신하지 않는다.

| 새로운 경로가 더 비싸면 갱신하지 않는다

$\text{dist}[2] = 3$



$$3 + 2 = 5$$

새 비용

5

>

기존 비용

2

$\text{dist}[3]$

2 = 2

갱신하지 않음

정점 3은 이미 비용 2로 확정했다. 나중에 비교한 $3+2=5$ 로는 갱신하지 않는다.

| 새로운 경로가 더 비싸면 갱신하지 않는다

$\text{dist}[2] = 3$



$$3 + 2 = 5$$

새 비용

5

>

기존 비용

2

$\text{dist}[3]$

2 = 2

갱신하지 않음

정점 3은 이미 비용 2로 확정했다. 나중에 비교한 $3+2=5$ 로는 갱신하지 않는다.

| 최소 비용 후보를 확정할 수 있는 이유

PQ에서 꺼낸 유효한 최소 후보

1 → 3

2

꺼낸 비용 = $\text{dist}[3] = 2$

정점 2를 거쳐 정점 3으로 간다면?

1 → 2까지 이미 비용 3

$$3 + (0 \text{ 이상}) \geq 3$$

2를 거치는 경로는 비용 2보다 작아질 수 없다.

실제 1 → 2 → 3의 비용도 $3 + 2 = 5$ 다.

유효한 최소 후보를 꺼낼 때, 그 정점의 비용을 확정한다.

모든 간선 비용 ≥ 0 이므로 먼저 확정할 수 있다.

나중에 2 → 3을 비교해도 $5 > 2$ 라서 갱신하지 않는다.

| 최소 비용 후보를 확정할 수 있는 이유

PQ에서 꺼낸 유효한 최소 후보

1 → 3

2

꺼낸 비용 = $\text{dist}[3] = 2$

정점 2를 거쳐 정점 3으로 간다면?

1 → 2까지 이미 비용 3

$$3 + (0 \text{ 이상}) \geq 3$$

2를 거치는 경로는 비용 2보다 작아질 수 없다.

실제 1 → 2 → 3의 비용도 $3 + 2 = 5$ 다.

유효한 최소 후보를 꺼낼 때, 그 정점의 비용을 확정한다.

모든 간선 비용 ≥ 0 이므로 먼저 확정할 수 있다.

나중에 2 → 3을 비교해도 $5 > 2$ 라서 갱신하지 않는다.

| 최소 비용 후보를 확정할 수 있는 이유

PQ에서 꺼낸 유효한 최소 후보

1 → 3

2

꺼낸 비용 = $\text{dist}[3] = 2$

정점 2를 거쳐 정점 3으로 간다면?

1 → 2까지 이미 비용 3

$$3 + (0 \text{ 이상}) \geq 3$$

2를 거치는 경로는 비용 2보다 작아질 수 없다.

실제 1 → 2 → 3의 비용도 $3 + 2 = 5$ 다.

유효한 최소 후보를 꺼낼 때, 그 정점의 비용을 확정한다.

모든 간선 비용 ≥ 0 이므로 먼저 확정할 수 있다.

나중에 2 → 3을 비교해도 $5 > 2$ 라서 갱신하지 않는다.

| PQ에서 꺼낸 비용을 현재 dist와 비교한다

PQ에 남은 정점 4의 기록만 확대

정점

누적 비용

4

5

← 최소

최신 비용표

dist[4] = 5

처음에는 비용 5인 경로를 발견했다.

PQ는 (정점, 넣을 당시 누적 비용)을 저장한다.

처음 발견한 비용 5를 PQ에 기록한다.

| PQ에서 꺼낸 비용을 현재 dist와 비교한다

PQ에 남은 정점 4의 기록만 확대

정점	누적 비용
4	4 ← 최소
4	5

PQ는 (정점, 넣을 당시 누적 비용)을 저장한다.

최신 비용표

dist[4] = 4

dist[4]를 4로 갱신한다.

새 후보 (4, 4)를 PQ에 추가한다.

기존 기록 (4, 5)도 그대로 남긴다.

dist를 갱신해도, PQ 안의 기존 기록을 찾아서 지우지는 않는다.

| PQ에서 꺼낸 비용을 현재 dist와 비교한다

PQ에 남은 정점 4의 기록만 확대

정점

누적 비용

4

5

← 최소

최신 비용표

dist[4] = 4

그 뒤, 비용 4인 후보를 먼저 꺼내 탐색했다.

정점 4의 남은 기록은 (4, 5)다.

PQ는 (정점, 넣을 당시 누적 비용)을 저장한다.

작은 비용의 후보부터 처리한다. 다음으로 (4, 5)를 꺼낼 차례다.

| PQ에서 꺼낸 비용을 현재 dist와 비교한다

PQ에 남은 정점 4의 기록만 확대

정점

누적 비용

정점 4의 기록 없음

최신 비용표

dist[4] = 4

이제 (4, 5)를 PQ에서 꺼냈다.

꺼낸 비용을 현재 dist[4]와 비교한다.

PQ는 (정점, 넣을 당시 누적 비용)을 저장한다.

poll() 결과: (4, 5)

currentDistance > distance[now] ?

다음 단계에서 5와 4를 비교한다.

| PQ에서 꺼낸 비용을 현재 dist와 비교한다

PQ에 남은 정점 4의 기록만 확대

정점

누적 비용

정점 4의 기록 없음

최신 비용표

dist[4] = 4

꺼낸 비용 5가 현재 dist[4]보다 크다.

dist는 그대로 두고 다음 후보로 넘어간다.

PQ는 (정점, 넣을 당시 누적 비용)을 저장한다.

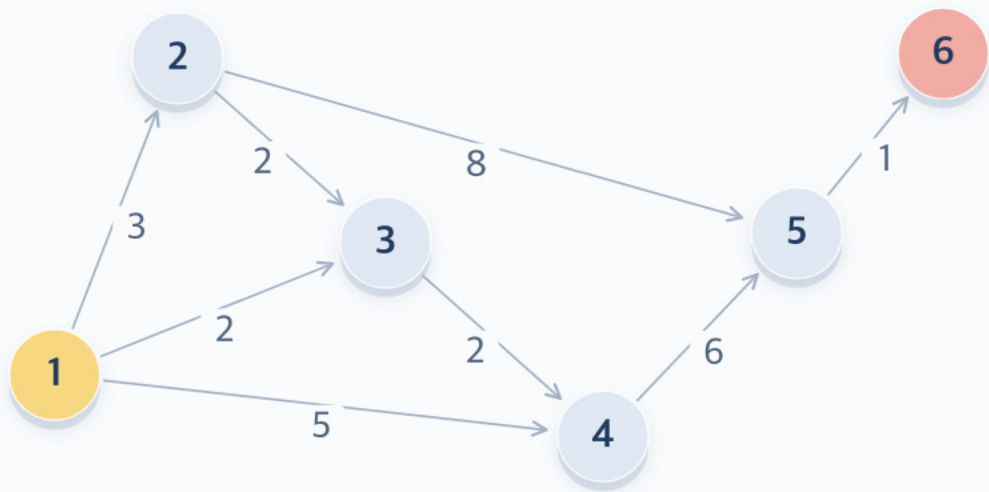
꺼낸 비용 5 > 현재 dist[4] = 4

```
if (currentDistance > distance[now]) continue;
```

오래된 후보 · 건너뛰기

| 배운 규칙으로 전체 실행 따라가기

dist와 PQ 초기화



	1	2	3	4	5	6
dist	0	∞	∞	∞	∞	∞

시작점의 비용은 0, 나머지는 ∞ 로 초기화한다.

• 초기화

현재 정점

Priority Queue

정점

누적 비용

1

0

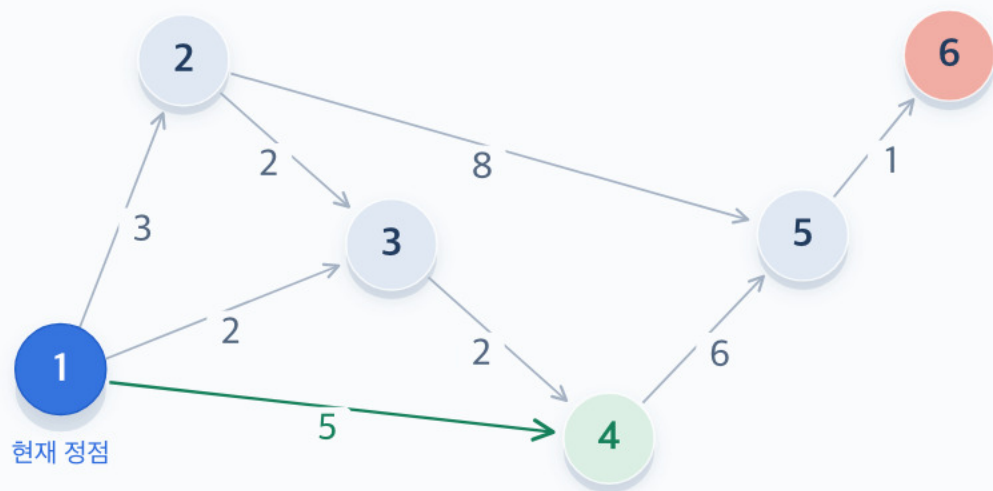
← 최소

시작점 초기화

`dist[start] = 0`

| 배운 규칙으로 전체 실행 따라가기

1번의 이웃 세 곳을 기록



	1	2	3	4	5	6
dist	0 확정	3	2	5	∞	∞

1번에서 이웃 2·3·4의 비용 3·2·5를 기록한다. 다음 최소 후보는?

• 거리 갱신

현재 정점

1

Priority Queue

정점	누적 비용	
3	2	← 최소
2	3	
4	5	

1 → 4

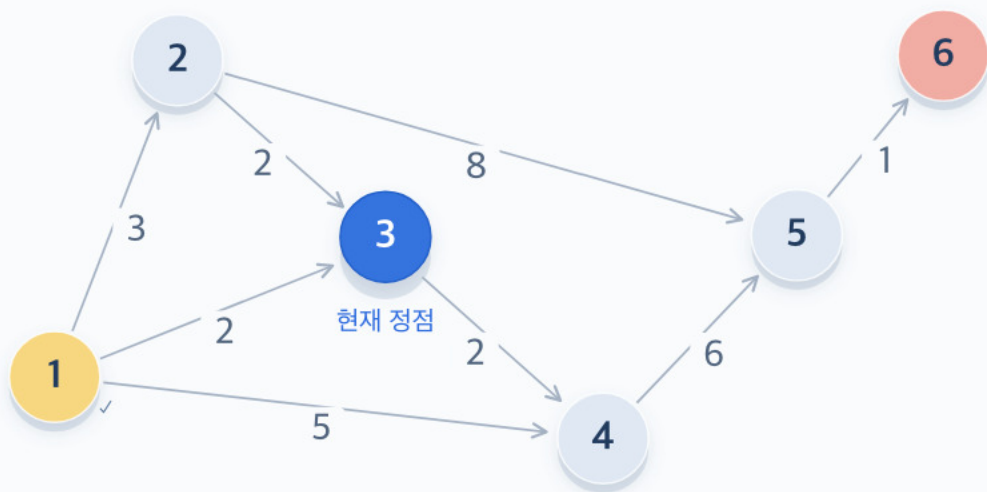
$$0 + 5 = 5$$

새 비용 5 < 기존 ∞

갱신 $\infty \rightarrow 5$

| 배운 규칙으로 전체 실행 따라가기

정점 3 선택·확정



	1	2	3	4	5	6
dist	0 확정	3	2 확정	5	∞	∞

다른 후보의 비용은 이미 3 이상이다. 음수 간선이 없으므로 $\text{dist}[3] = 2$ 를 확정한다.

• 최소 후보 선택

현재 정점

3

Priority Queue

정점

누적 비용

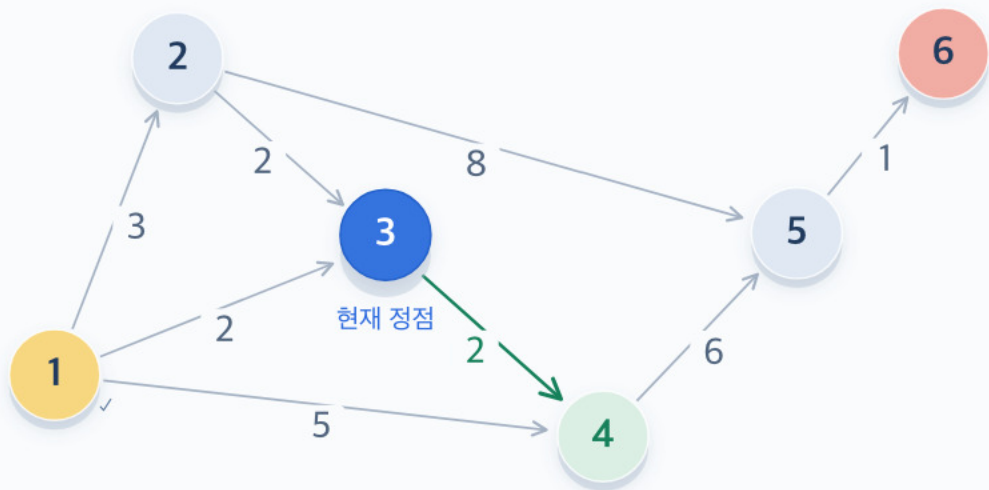
2	3	← 최소
4	5	

이 정점의 최소 비용 확정

2

| 배운 규칙으로 전체 실행 따라가기

dist[4] 갱신 + PQ 추가



dist	1	2	3	4	5	6
	0	3	2	4	∞	∞
	확정		확정			

더 싼 경로를 발견하면 dist를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 정점

3

Priority Queue

정점	누적 비용	
2	3	← 최소
4	4	
4	5	

3 → 4

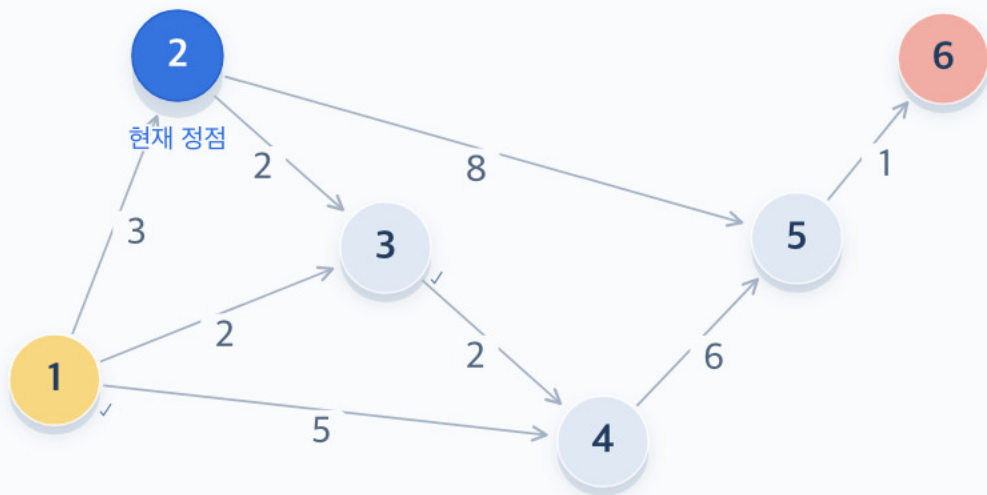
$$2 + 2 = 4$$

새 비용 4 < 기존 5

갱신 5 → 4

| 배운 규칙으로 전체 실행 따라가기

정점 2 선택·확정



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4	∞	∞

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 정점

2

Priority Queue

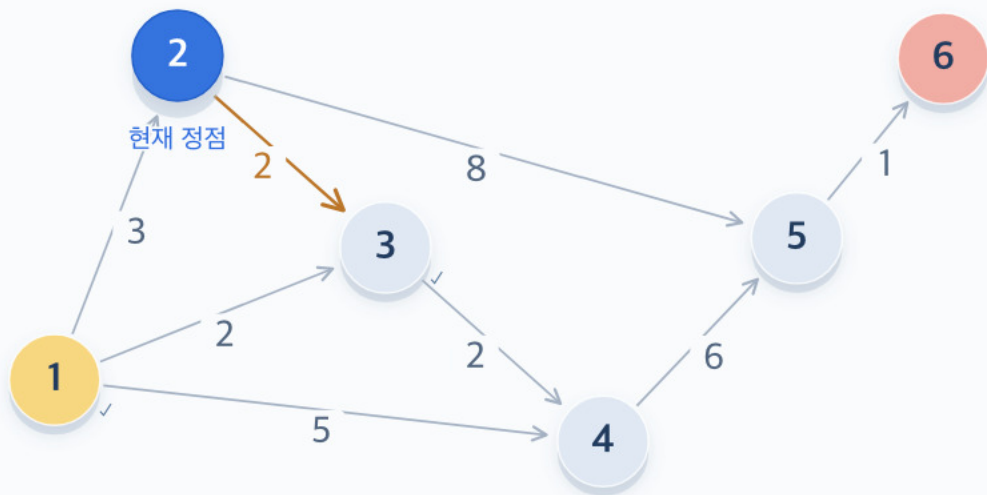
정점	누적 비용	
4	4	← 최소
4	5	

이 정점의 최소 비용 확정

3

| 배운 규칙으로 전체 실행 따라가기

dist[3] 유지



	1	2	3	4	5	6
dist	0	3	2	4	∞	∞
	확정	확정	확정			

나중에 2→3을 비교해도 $3+2=5 > 2$ 이므로, 이미 확정된 dist[3]은 그대로다.

- 갱신하지 않음

현재 정점

2

Priority Queue

정점	누적 비용	
4	4	← 최소
4	5	

2 → 3

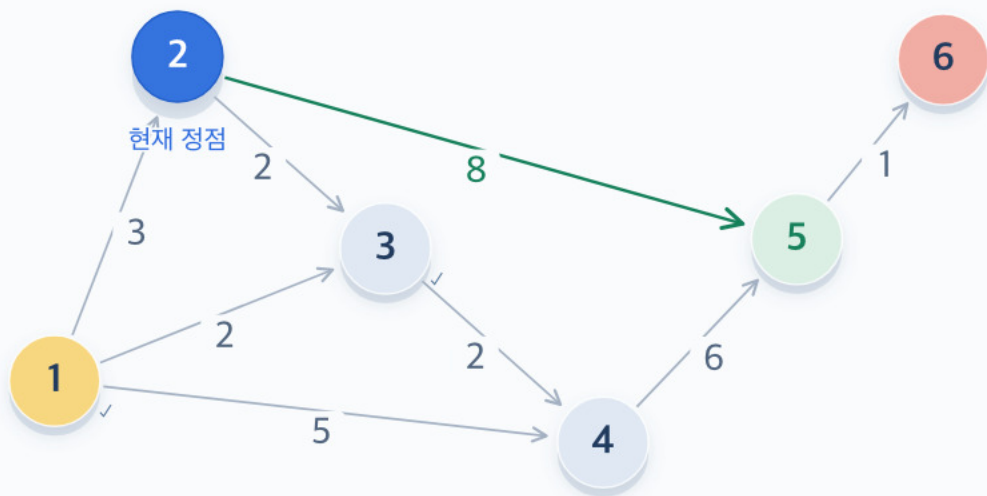
$$3 + 2 = 5$$

새 비용 5 > 기존 2

갱신하지 않음

| 배운 규칙으로 전체 실행 따라가기

dist[5] 갱신 + PQ 추가



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4	11	∞

더 싼 경로를 발견하면 dist를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 정점

2

Priority Queue

정점	누적 비용	
4	4	← 최소
4	5	
5	11	

2 → 5

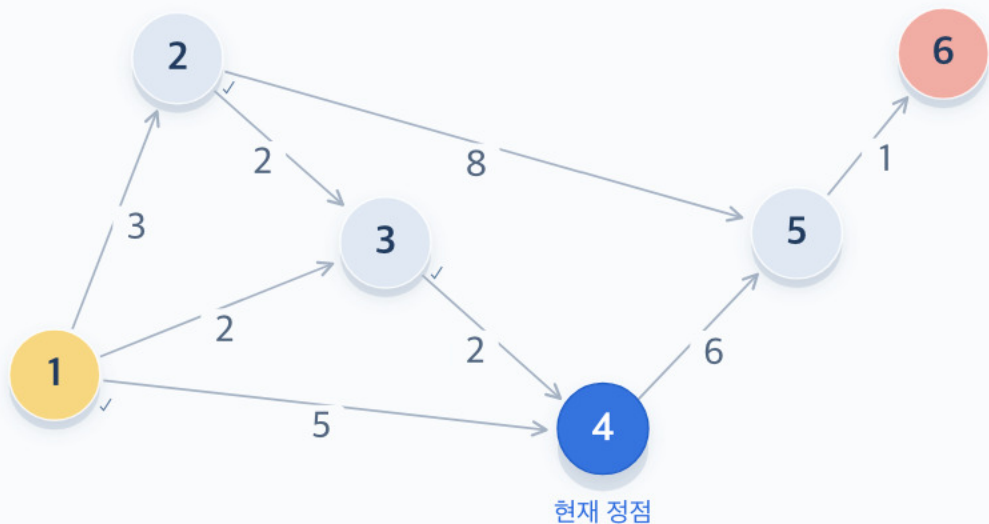
$$3 + 8 = 11$$

새 비용 11 < 기존 ∞

갱신 ∞ → 11

| 배운 규칙으로 전체 실행 따라가기

정점 4 선택·확정



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	11	∞

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 정점

4

Priority Queue

정점

누적 비용

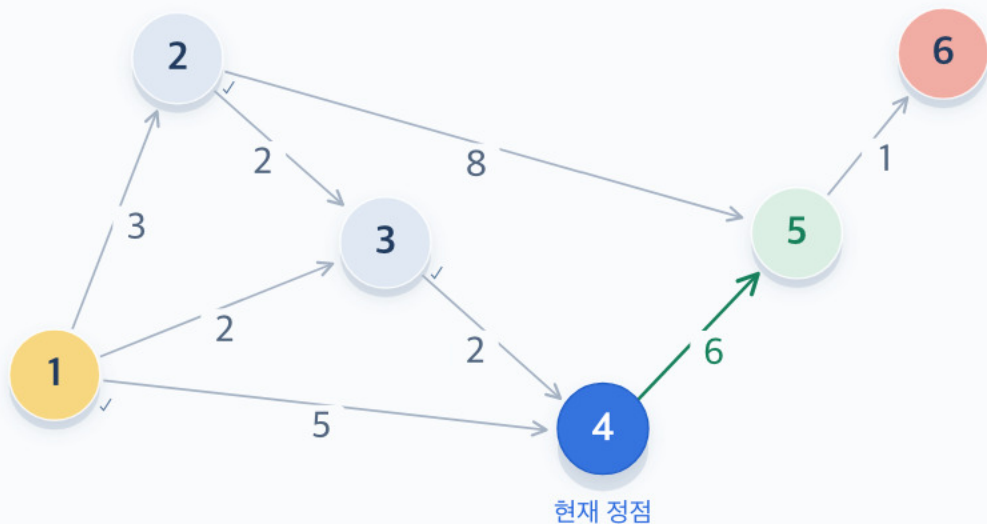
4	5	← 최소
5	11	

이 정점의 최소 비용 확정

4

| 배운 규칙으로 전체 실행 따라가기

dist[5] 갱신 + PQ 추가



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10	∞

더 싼 경로를 발견하면 dist를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 정점

4

Priority Queue

정점	누적 비용	
4	5	← 최소
5	10	
5	11	

4 → 5

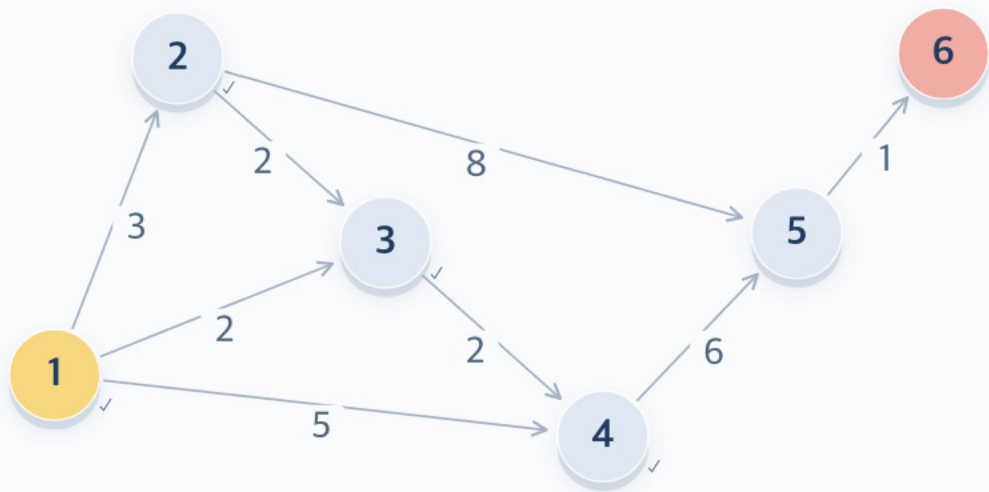
$$4 + 6 = 10$$

새 비용 10 < 기존 11

갱신 11 → 10

| 배운 규칙으로 전체 실행 따라가기

후보 (4, 5) 꺼내기



	1	2	3	4	5	6
dist	0	3	2	4	10	∞
	확정	확정	확정	확정		

PQ에서 후보를 꺼냈다. 이제 꺼낸 비용을 현재 dist와 비교한다.

• 후보 꺼내기

꺼낸 후보

4

Priority Queue

정점

누적 비용

5	10	← 최소
5	11	

poll()로 꺼낸 비용

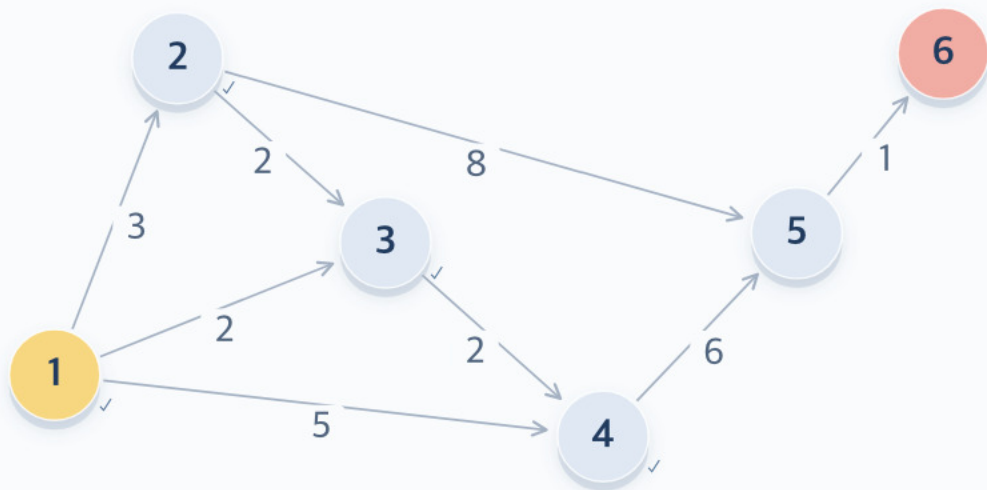
5

현재 dist = 4

다음: 두 비용 비교

| 배운 규칙으로 전체 실행 따라가기

비용 비교 후 오래된 기록 건너뛰기



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10	∞

현재 dist보다 큰 오래된 후보는 건너뛴다.

• 오래된 후보

꺼낸 후보

4

Priority Queue

정점

누적 비용

5	10	← 최소
5	11	

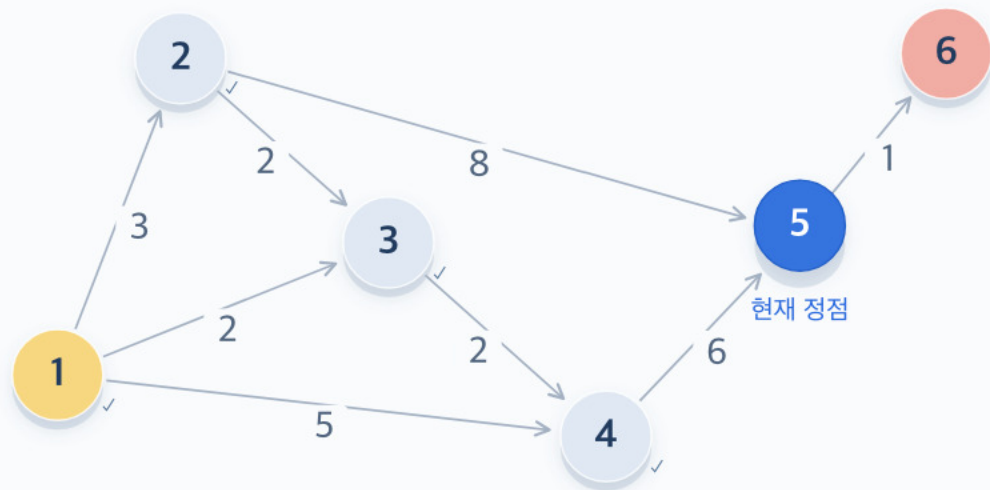
꺼낸 비용 > 현재 최소 비용

5 > 4

오래된 후보 · 건너뛰기

| 배운 규칙으로 전체 실행 따라가기

정점 5 선택·확정



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10 확정	∞

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 정점

5

Priority Queue

정점

누적 비용

5

11

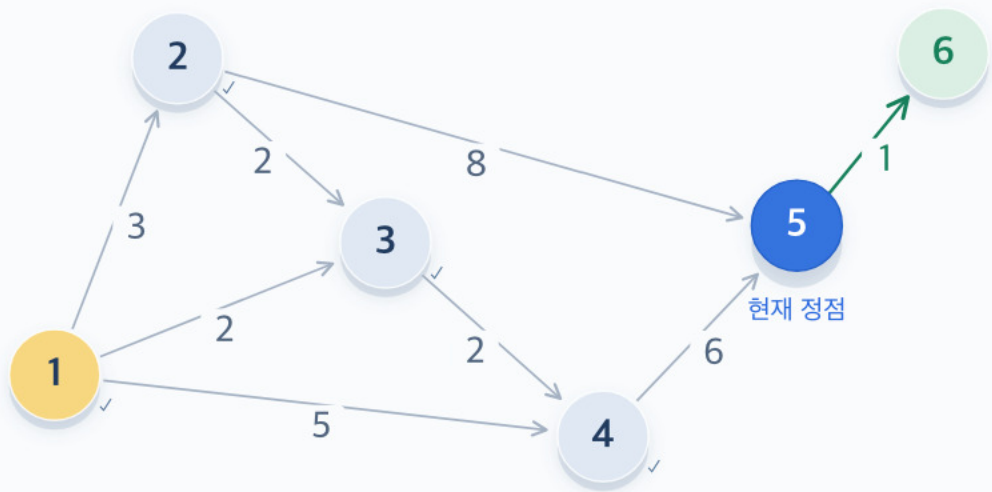
← 최소

이 정점의 최소 비용 확정

10

| 배운 규칙으로 전체 실행 따라가기

dist[6] 갱신 + PQ 추가



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10 확정	11

더 싼 경로를 발견하면 dist를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 정점

5

Priority Queue

정점	누적 비용	
5	11	← 최소
6	11	

5 → 6

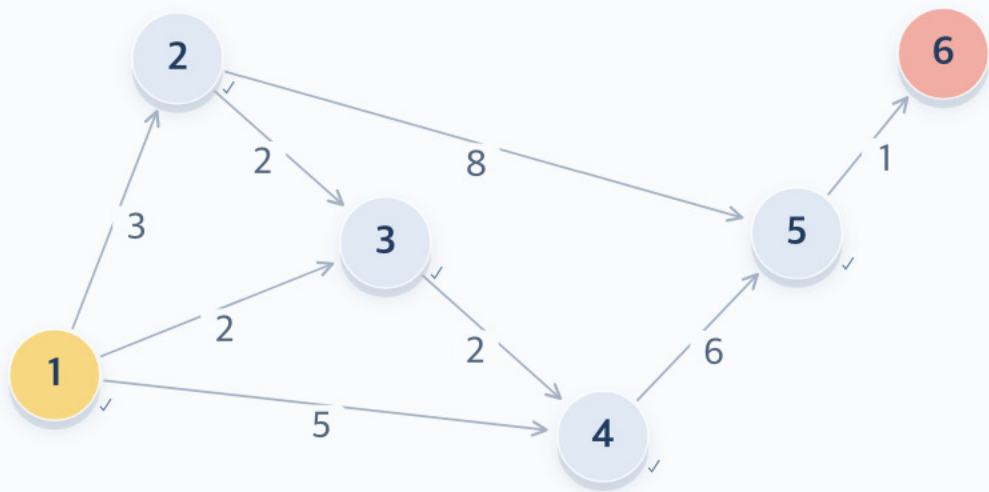
$$10 + 1 = 11$$

새 비용 11 < 기존 ∞

갱신 ∞ → 11

| 배운 규칙으로 전체 실행 따라가기

후보 (5, 11) 꺼내기



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10 확정	11

• 후보 꺼내기

꺼낸 후보

5

Priority Queue

정점

누적 비용

6

11

← 최소

poll()로 꺼낸 비용

11

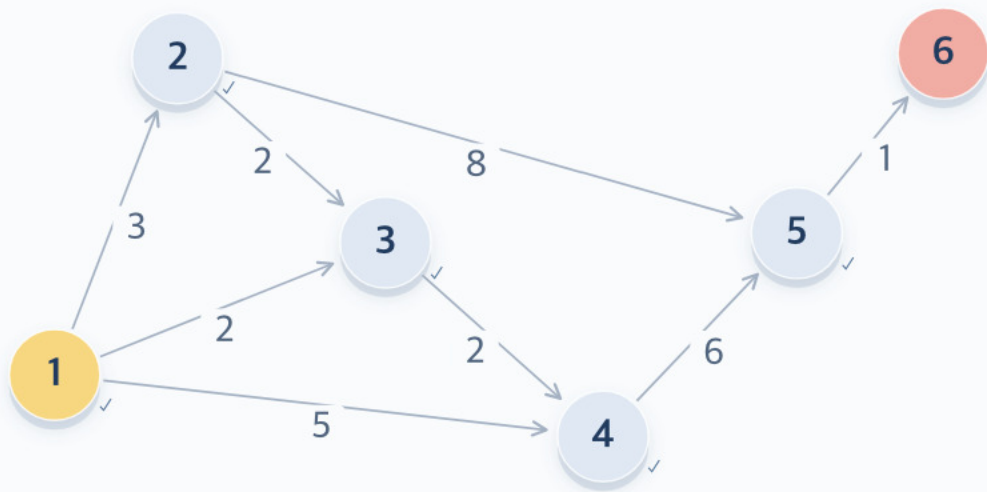
현재 dist = 10

다음: 두 비용 비교

PQ에서 후보를 꺼냈다. 이제 꺼낸 비용을 현재 dist와 비교한다.

| 배운 규칙으로 전체 실행 따라가기

비용 비교 후 오래된 기록 건너뛰기



	1	2	3	4	5	6
dist	0 확정	3 확정	2 확정	4 확정	10 확정	11

현재 dist보다 큰 오래된 후보는 건너뛴다.

• 오래된 후보

꺼낸 후보

5

Priority Queue

정점

누적 비용

6

11

← 최소

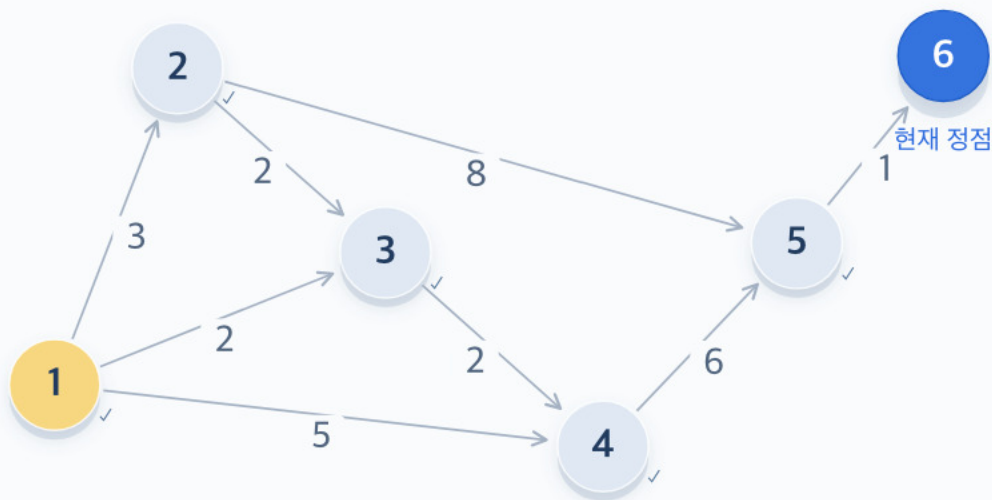
꺼낸 비용 > 현재 최소 비용

11 > 10

오래된 후보 · 건너뛰기

| 배운 규칙으로 전체 실행 따라가기

정점 6 선택·확정



	1	2	3	4	5	6
dist	0	3	2	4	10	11
	확정	확정	확정	확정	확정	확정

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 정점

6

Priority Queue

정점

누적 비용

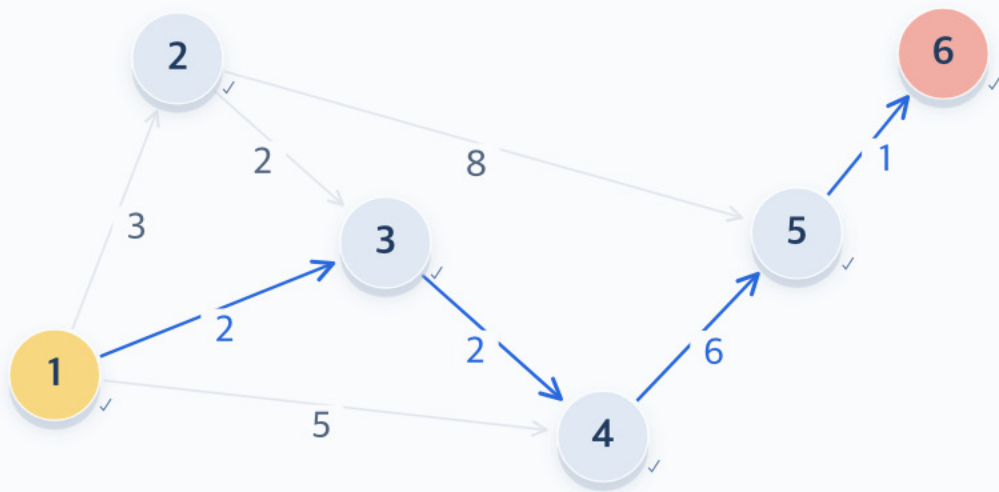
비어 있음

이 정점의 최소 비용 확정

11

| 배운 규칙으로 전체 실행 따라가기

최종 비용 확인



	1	2	3	4	5	6
dist	0	3	2	4	10	11
	확정	확정	확정	확정	확정	확정

도달 가능한 정점의 최소 비용이 확정되었다.

• 탐색 완료

현재 정점

Priority Queue

정점

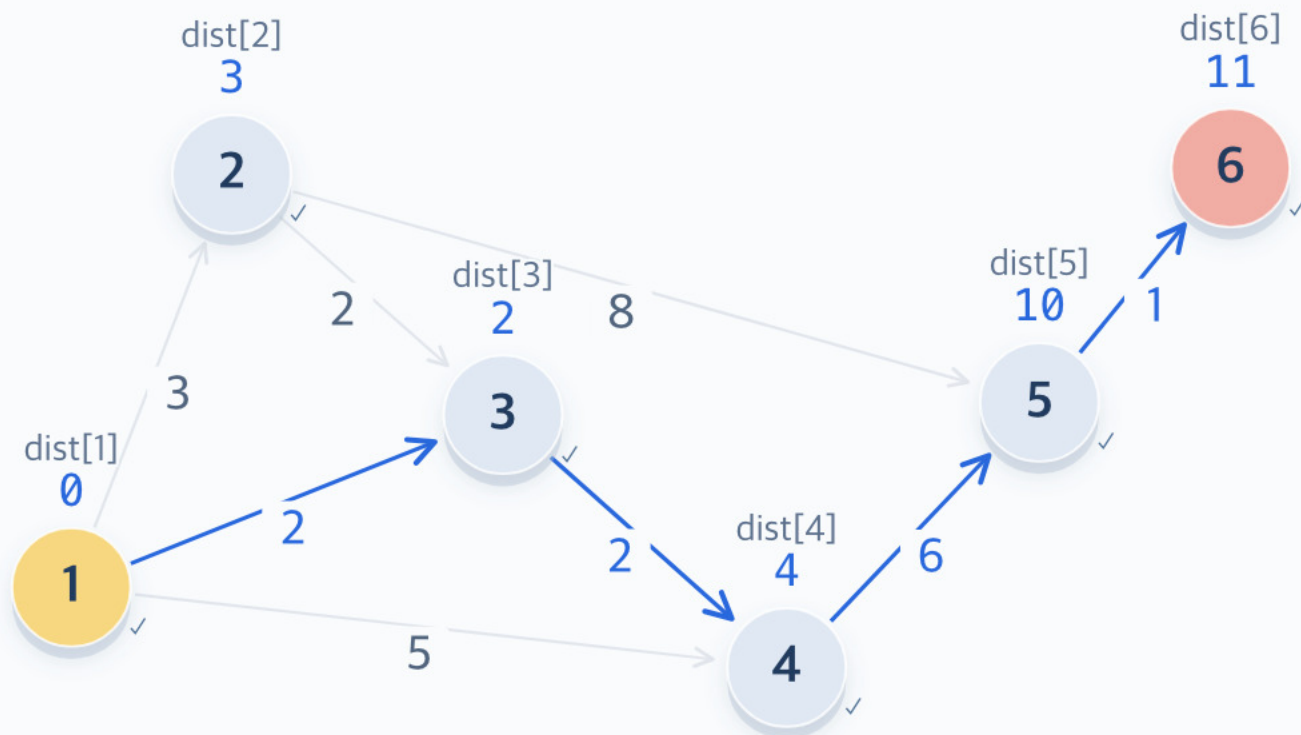
누적 비용

비어 있음

모든 유효 후보 탐색 완료

PQ = ∅

| 1에서 6까지의 최소 비용은 11이다



최소 비용 경로

1 → 3 → 4 → 5 → 6

2 + 2 + 6 + 1 =

11

| Java 코드로 보는 다익스트라

Java

1 / 3 · 그래프 구성

```
01 List<int[]>[] graph = new ArrayList[n + 1];
02 for (int i = 1; i <= n; i++) {
03     graph[i] = new ArrayList<>();
04 }
06 for (int[] road : roads) {
07     int from = road[0];
08     int to = road[1];
09     int cost = road[2];
10     graph[from].add(new int[] {to, cost});
11 }
```

01

방향 그래프 구성

graph[from]에 {to, cost}

각 간선을 출발 정점의 리스트에 넣는다.
반대 방향은 추가하지 않는다.

solution(n, start, roads) 핵심 발췌

| Java 코드로 보는 다익스트라

Java

2 / 3 · distance와 PQ 초기화

```
01 int[] distance = new int[n + 1];
02 Arrays.fill(distance, Integer.MAX_VALUE);
03 distance[start] = 0;

05 PriorityQueue<int[]> pq = new PriorityQueue<>(
06     (a, b) -> Integer.compare(a[1], b[1]));
07 pq.offer(new int[] {start, 0});
```

02

최소 비용표 초기화

`dist = distance`

앞서 본 dist 배열이다. 시작점은 0, 나머지는 Integer.MAX_VALUE로 초기화한다.

`solution(n, start, roads)` 핵심 발췌

| Java 코드로 보는 다익스트라

Java

2 / 3 · distance와 PQ 초기화

```
01 int[] distance = new int[n + 1];
02 Arrays.fill(distance, Integer.MAX_VALUE);
03 distance[start] = 0;
05 PriorityQueue<int[]> pq = new PriorityQueue<>(
06     (a, b) -> Integer.compare(a[1], b[1]));
07 pq.offer(new int[] {start, 0});
```

03

누적 비용 순으로 정렬

pq: {정점, 누적 비용}

배열의 [1]을 비교한다. 시작 후보 {start, 0}을 넣는다.

solution(n, start, roads) 핵심 발췌

| Java 코드로 보는 다익스트라

Java

3 / 3 · 탐색 반복

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int now = current[0];
04     int currentDistance = current[1];
05     if (currentDistance > distance[now]) {
06         continue;
07     }
08     for (int[] next : graph[now]) {
09         int nextNode = next[0];
10         int cost = next[1];
11         int newDistance = currentDistance + cost;
12         if (newDistance < distance[nextNode]) {
13             distance[nextNode] = newDistance;
14             pq.offer(new int[] {
15                 nextNode, newDistance});
16         }
17     }
18 }
19 }
```

04

최소 후보 꺼내기

`now` · `currentDistance`

`current[0]`은 정점, `current[1]`은 PQ에 넣을 당시의 누적 비용이다.

`solution(n, start, roads)` 핵심 발췌

| Java 코드로 보는 다익스트라

Java

3 / 3 · 탐색 반복

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int now = current[0];
04     int currentDistance = current[1];
05     if (currentDistance > distance[now]) {
06         continue;
07     }
08     for (int[] next : graph[now]) {
09         int nextNode = next[0];
10         int cost = next[1];
11         int newDistance = currentDistance + cost;
12         if (newDistance < distance[nextNode]) {
13             distance[nextNode] = newDistance;
14             pq.offer(new int[] {
15                 nextNode, newDistance});
16         }
17     }
18 }
19 }
```

05

꺼낸 비용 비교

`currentDistance >`
`distance[now]`

더 크면 오래된 후보이므로 건너뛴다. 이
검사를 통과한 최소 후보의 거리는 확정된다.

`solution(n, start, roads)` 핵심 발췌

| Java 코드로 보는 다익스트라

Java

3 / 3 · 탐색 반복

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int now = current[0];
04     int currentDistance = current[1];
05     if (currentDistance > distance[now]) {
06         continue;
07     }
08     for (int[] next : graph[now]) {
09         int nextNode = next[0];
10         int cost = next[1];
11         int newDistance = currentDistance + cost;
12         if (newDistance < distance[nextNode]) {
13             distance[nextNode] = newDistance;
14             pq.offer(new int[] {
15                 nextNode, newDistance});
16         }
17     }
18 }
19 }
```

06

인접 정점 확인

next: {다음 정점, 간선 비용}

graph[now]를 순회한다. next[1]은 누적 비용이 아닌 한 간선의 비용이다.

solution(n, start, roads) 핵심 발췌

| Java 코드로 보는 다익스트라

Java

3 / 3 · 탐색 반복

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int now = current[0];
04     int currentDistance = current[1];
05     if (currentDistance > distance[now]) {
06         continue;
07     }
08     for (int[] next : graph[now]) {
09         int nextNode = next[0];
10         int cost = next[1];
11         int newDistance = currentDistance + cost;
12         if (newDistance < distance[nextNode]) {
13             distance[nextNode] = newDistance;
14             pq.offer(new int[] {
15                 nextNode, newDistance});
16         }
17     }
18 }
19 }
```

07

새 비용 계산·비교

`currentDistance + cost`

newDistance를 계산하고,
distance[nextNode]보다 작은지 비교한다.

`solution(n, start, roads)` 핵심 발췌

| Java 코드로 보는 다익스트라

Java

3 / 3 · 탐색 반복

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int now = current[0];
04     int currentDistance = current[1];
05     if (currentDistance > distance[now]) {
06         continue;
07     }
08     for (int[] next : graph[now]) {
09         int nextNode = next[0];
10         int cost = next[1];
11         int newDistance = currentDistance + cost;
12         if (newDistance < distance[nextNode]) {
13             distance[nextNode] = newDistance;
14             pq.offer(new int[] {
15                 nextNode, newDistance});
16         }
17     }
18 }
19 }
```

08

갱신하고 PQ에 추가

{nextNode, newDistance}

더 작은 비용을 distance에 기록하고 새 후보를 넣는다. PQ에 남은 이전 기록은 그대로 둔다.

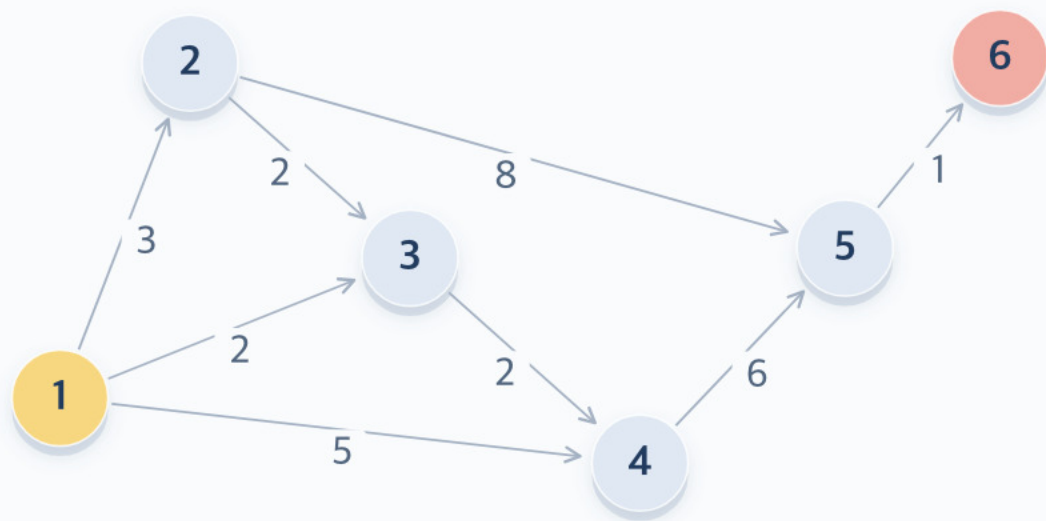
solution(n, start, roads) 핵심 발췌

04

보급로에 적용

정점과 간선을 좌표와 상하좌우 이동으로 바꾸고,
코드와 격자 실행으로 최소 복구 시간을 확인합니다.

| 그럼 이걸 보급로에 적용하면 어떻게 될까?



Node → (x, y)

Edge → 상 / 하 / 좌 / 우

Weight → map[nextX][nextY]

distance[v] → minCost[x][y]

| 그럼 이걸 보급로에 적용하면 어떻게 될까?

Node → (x, y)

Edge → 상 / 하 / 좌 / 우

Weight → map[nextX][nextY]

distance[v] → minCost[x][y]

| 그럼 이걸 보급로에 적용하면 어떻게 될까?

Node → (x, y)

Edge → 상 / 하 / 좌 / 우

Weight → map[nextX][nextY]

distance[v] → minCost[x][y]

S (0, 0)

G (N-1, N-1)

| 그럼 이걸 보급로에 적용하면 어떻게 될까?

Node → (x, y)

Edge → 상 / 하 / 좌 / 우

Weight → map[nextX][nextY]

distance[v] → minCost[x][y]

S (0, 0)

G (N-1, N-1)

| 그럼 이걸 보급로에 적용하면 어떻게 될까?

Node → (x, y)

Edge → 상 / 하 / 좌 / 우

Weight → map[nextX][nextY]

distance[v] → minCost[x][y]

S (0, 0)

G (N-1, N-1)

| 좌표가 정점, 이동할 칸의 값이 가중치다

	0	1	2
0	S 0	1	5
1	2	1 현재 위치	2
2	4	1	G 0

숫자 = 해당 칸의 복구 시간

정점 · 선택한 칸

(1, 1)

간선 상 / 하 / 좌 / 우

선택한 칸의 값

`map[1][1] = 1`

이동할 때 더하는 비용

`currentCost + map[nextX][nextY]`

다음 칸에 들어가는 비용을 더한다.

| 좌표가 정점, 이동할 칸의 값이 가중치다

	0	1	2
0	S 0	1	5
1	2	1 현재 위치	2
2	4	1	G 0

숫자 = 해당 칸의 복구 시간

정점 · 선택한 칸

(1, 1)

간선 상 / 하 / 좌 / 우

선택한 칸의 값

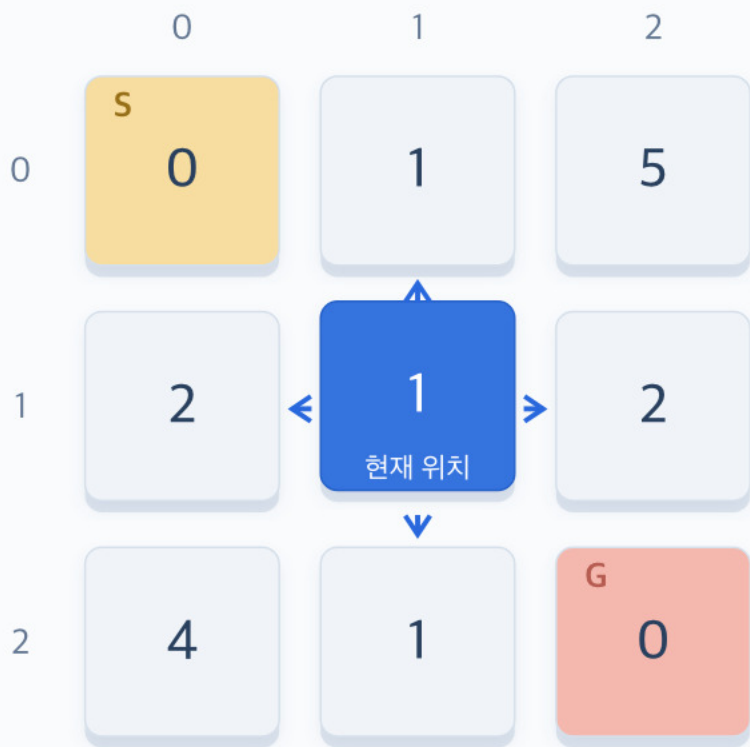
`map[1][1] = 1`

이동할 때 더하는 비용

`currentCost + map[nextX][nextY]`

다음 칸에 들어가는 비용을 더한다.

| 좌표가 정점, 이동할 칸의 값이 가중치다



숫자 = 해당 칸의 복구 시간

정점 · 선택한 칸

(1, 1)

간선 상 / 하 / 좌 / 우

선택한 칸의 값

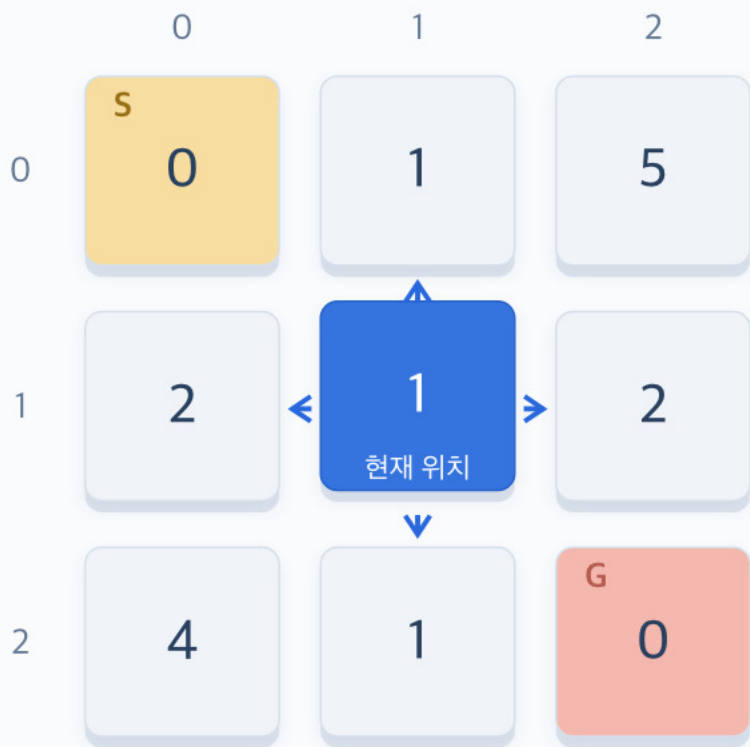
`map[1][1] = 1`

이동할 때 더하는 비용

`currentCost + map[nextX][nextY]`

다음 칸에 들어가는 비용을 더한다.

| 좌표가 정점, 이동할 칸의 값이 가중치다



숫자 = 해당 칸의 복구 시간

정점 · 선택한 칸

(1, 1)

간선 상 / 하 / 좌 / 우

선택한 칸의 값

`map[1][1] = 1`

이동할 때 더하는 비용

`currentCost + map[nextX][nextY]`

다음 칸에 들어가는 비용을 더한다.

| Java 코드로 보는 보급로

Java

1 / 3 · 좌표와 초기화

```
01 int[] dx = {-1, 1, 0, 0};
02 int[] dy = {0, 0, -1, 1};
03 Queue<int[]> pq = new PriorityQueue<>(
04     (a, b) -> Integer.compare(a[2], b[2]));

06 int[][] minCost = new int[N][N];
07 for (int i = 0; i < N; i++) {
08     Arrays.fill(minCost[i], Integer.MAX_VALUE);
09 }
10 minCost[0][0] = 0;
11 pq.offer(new int[] {0, 0, 0});
```

01

좌표와 우선순위 큐

{x, y, 누적 비용}

x는 행, y는 열이다. PQ는 배열의 [2]를
비교해 누적 비용이 작은 후보부터 꺼낸다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

1 / 3 · 좌표와 초기화

```
01 int[] dx = {-1, 1, 0, 0};
02 int[] dy = {0, 0, -1, 1};
03 Queue<int[]> pq = new PriorityQueue<>(
04     (a, b) -> Integer.compare(a[2], b[2]));
06 int[][] minCost = new int[N][N];
07 for (int i = 0; i < N; i++) {
08     Arrays.fill(minCost[i], Integer.MAX_VALUE);
09 }
10 minCost[0][0] = 0;
11 pq.offer(new int[] {0, 0, 0});
```

02

최소 비용표 초기화

distance[v]

minCost[x][y]

정점별 거리 배열이 2차원으로 바뀐다. 시작 칸은 0, 나머지는 무한대로 초기화한다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

2 / 3 · 최소 후보와 종료 조건

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int currentX = current[0];
04     int currentY = current[1];
05     int currentCost = current[2];
06     if (currentCost > minCost[currentX][currentY]) {
07         continue;
08     }
09
10     if (currentX == N - 1 && currentY == N - 1) {
11         break;
12     }
13     // 이어서 상하좌우 탐색
```

03

최소 후보 꺼내기

currentX, currentY
currentCost

current[0], [1]은 좌표이고 [2]는 PQ에 넣을
당시의 누적 복구 비용이다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

2 / 3 · 최소 후보와 종료 조건

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int currentX = current[0];
04     int currentY = current[1];
05     int currentCost = current[2];
06     if (currentCost > minCost[currentX][currentY]) {
07         continue;
08     }
10     if (currentX == N - 1 && currentY == N - 1) {
11         break;
12     }
13     // 이어서 상하좌우 탐색
```

04

오래된 후보 검사

`currentCost >`
`minCost[currentX][currentY]`

꺼낸 비용이 최신 비용보다 크면 건너뛴다.
목적지 확인도 이 검사 다음에 한다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

2 / 3 · 최소 후보와 종료 조건

```
01 while (!pq.isEmpty()) {
02     int[] current = pq.poll();
03     int currentX = current[0];
04     int currentY = current[1];
05     int currentCost = current[2];
06     if (currentCost > minCost[currentX][currentY]) {
07         continue;
08     }
09
10     if (currentX == N - 1 && currentY == N - 1) {
11         break;
12     }
13     // 이어서 상하좌우 탐색
```

05

목적지에서 종료

 $(N - 1, N - 1)$

목적지를 유효한 최소 후보로 꺼냈으므로 비용이 확정된다. PQ에 후보가 남아 있어도 종료한다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

3 / 3 · while문 안의 상하좌우 탐색

```

01     for (int dir = 0; dir < 4; dir++) {
02         int nextX = currentX + dx[dir];
03         int nextY = currentY + dy[dir];
04         if (nextX < 0 || nextX >= N ||
05             nextY < 0 || nextY >= N) {
06             continue;
07         }
08         int cost = map[nextX][nextY];
09         int newDistance = currentCost + cost;
10         if (newDistance < minCost[nextX][nextY]) {
11             minCost[nextX][nextY] = newDistance;
12             pq.offer(new int[] {
13                 nextX, nextY, newDistance});
14         }
15     }
16 }

```

06

상하좌우 탐색

currentX + dx[dir]

currentY + dy[dir]

인접 리스트 대신 방향 배열로 이웃을
구한다. 지도 밖의 좌표는 제외한다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

3 / 3 · while문 안의 상하좌우 탐색

```
01     for (int dir = 0; dir < 4; dir++) {
02         int nextX = currentX + dx[dir];
03         int nextY = currentY + dy[dir];
04         if (nextX < 0 || nextX >= N ||
05             nextY < 0 || nextY >= N) {
06             continue;
07         }
08         int cost = map[nextX][nextY];
09         int newDistance = currentCost + cost;
10         if (newDistance < minCost[nextX][nextY]) {
11             minCost[nextX][nextY] = newDistance;
12             pq.offer(new int[] {
13                 nextX, nextY, newDistance});
14         }
15     }
16 }
```

07

이동할 칸의 비용

 $currentCost + cost$

cost는 map[nextX][nextY]다. 다음 칸에 들어가는 비용을 더해 기존 최소 비용과 비교한다.

SWEA 1249 핵심 발췌

| Java 코드로 보는 보급로

Java

3 / 3 · while문 안의 상하좌우 탐색

```
01     for (int dir = 0; dir < 4; dir++) {
02         int nextX = currentX + dx[dir];
03         int nextY = currentY + dy[dir];
04         if (nextX < 0 || nextX >= N ||
05             nextY < 0 || nextY >= N) {
06             continue;
07         }
08         int cost = map[nextX][nextY];
09         int newDistance = currentCost + cost;
10         if (newDistance < minCost[nextX][nextY]) {
11             minCost[nextX][nextY] = newDistance;
12             pq.offer(new int[] {
13                 nextX, nextY, newDistance});
14         }
15     }
16 }
```

08

갱신하고 PQ에 추가

`{nextX, nextY, newDistance}`

더 작은 값을 minCost에 기록하고 새 후보를 넣는다. 최종 답은 minCost[N-1][N-1]이다.

SWEA 1249 핵심 발췌

| 같은 다익스트라를 격자에서 실행한다

	0	1	2
0	S 칸 비용 0 0	칸 비용 1 ∞	칸 비용 5 ∞
1	칸 비용 2 ∞	칸 비용 1 ∞	칸 비용 2 ∞
2	칸 비용 4 ∞	칸 비용 1 ∞	G 칸 비용 0 ∞

큰 숫자 = 현재 minCost

시작점의 비용은 0, 나머지는 ∞ 로 초기화한다.

• 초기화

현재 좌표

Priority Queue

좌표

누적 비용

(0, 0)

0

← 최소

시작점 초기화

$\text{minCost}[0][0] = 0$

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

- 최소 후보 선택

현재 좌표 (0, 0)

Priority Queue

좌표 누적 비용

비어 있음

이 정점의 최소 비용 확정

0

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (0, 0)

Priority Queue

좌표	누적 비용	
(1, 0)	2	← 최소

(0, 0) → (1, 0)

$$0 + 2 = 2$$

새 비용 2 < 기존 ∞

갱신 $\infty \rightarrow 2$

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (0, 0)

Priority Queue

좌표	누적 비용	
(0, 1)	1	← 최소
(1, 0)	2	

(0, 0) → (0, 1)

$$0 + 1 = 1$$

새 비용 1 < 기존 ∞

갱신 $\infty \rightarrow 1$

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

• 최소 후보 선택

현재 좌표 (0, 1)

Priority Queue

좌표

누적 비용

(1, 0)

2

← 최소

이 정점의 최소 비용 확정

1

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

| 같은 다익스트라를 격자에서 실행한다



• 거리 갱신

현재 좌표 (0, 1)

Priority Queue

좌표	누적 비용	
(1, 0)	2	← 최소
(1, 1)	2	

(0, 1) → (1, 1)

$$1 + 1 = 2$$

새 비용 2 < 기존 ∞

갱신 ∞ → 2

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (0, 1)

Priority Queue

좌표	누적 비용	
(1, 0)	2	← 최소
(1, 1)	2	
(0, 2)	6	

(0, 1) → (0, 2)

$$1 + 5 = 6$$

새 비용 6 < 기존 ∞

갱신 ∞ → 6

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

• 최소 후보 선택

현재 좌표 (1, 0)

Priority Queue

좌표	누적 비용	
(1, 1)	2	← 최소
(0, 2)	6	

이 정점의 최소 비용 확정

2

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (1, 0)

Priority Queue

좌표	누적 비용	
(1, 1)	2	← 최소
(0, 2)	6	
(2, 0)	6	

(1, 0) → (2, 0)

$$2 + 4 = 6$$

새 비용 6 < 기존 ∞

갱신 ∞ → 6

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 좌표 (1, 1)

Priority Queue

좌표	누적 비용	
(0, 2)	6	← 최소
(2, 0)	6	

이 정점의 최소 비용 확정

2

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (1, 1)

Priority Queue

좌표	누적 비용	
(2, 1)	3	← 최소
(0, 2)	6	
(2, 0)	6	

(1, 1) → (2, 1)

$$2 + 1 = 3$$

새 비용 3 < 기존 ∞

갱신 ∞ → 3

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 (1, 1)

Priority Queue

좌표	누적 비용	
(2, 1)	3	← 최소
(1, 2)	4	
(0, 2)	6	
(2, 0)	6	

(1, 1) → (1, 2)

$$2 + 2 = 4$$

새 비용 4 < 기존 ∞

갱신 ∞ → 4

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 좌표 (2, 1)

Priority Queue

좌표	누적 비용	
(1, 2)	4	← 최소
(0, 2)	6	
(2, 0)	6	

이 정점의 최소 비용 확정

3

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

더 싼 경로를 발견하면 minCost를 갱신하고 PQ에 추가한다.

• 거리 갱신

현재 좌표 **(2, 1)**

Priority Queue

좌표	누적 비용	
(2, 2)	3	← 최소
(1, 2)	4	
(0, 2)	6	
(2, 0)	6	

(2, 1) → (2, 2)

3 + 0 = 3

새 비용 3 < 기존 ∞

갱신 ∞ → 3

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

PQ에서 누적 비용이 가장 작은 후보를 꺼낸다.

• 최소 후보 선택

현재 좌표 (2, 2)

Priority Queue

좌표	누적 비용	
(1, 2)	4	← 최소
(0, 2)	6	
(2, 0)	6	

이 정점의 최소 비용 확정

3

| 같은 다익스트라를 격자에서 실행한다



큰 숫자 = 현재 minCost

• 탐색 완료

현재 좌표 (2, 2)

Priority Queue

좌표	누적 비용	
(1, 2)	4	← 최소
(0, 2)	6	
(2, 0)	6	

목적지 최소 비용 확정

3

PQ에 후보가 남아도 종료

목적지 (2,2)의 최소 복구 비용 3이 확정됐다. PQ에 후보가 남아 있어도 종료한다.

| 목적지의 minCost가 최소 복구 시간이다



큰 숫자 = 현재 minCost

최소 복구 시간

$\text{minCost}[N-1][N-1]$

3

$$1 + 1 + 1 + 0 = 3$$

복구 경로

$(0, 0) \rightarrow (0, 1) \rightarrow (1, 1) \rightarrow$

$(2, 1) \rightarrow (2, 2)$

05

정리

문제의 비용 조건에 따른 알고리즘 선택 기준을 정리하고,
보급로와 다익스트라의 핵심 원리를 되짚습니다.

| 이런 조건이라면 다익스트라를 고려한다

01 최소 비용 / 최소 시간 / 최단 거리? ✓

02 그래프로 표현할 수 있는가? ✓

03 이동마다 서로 다른 비용이 있는가? ✓

04 음수 가중치가 없는가? ✓

조건으로 판단

Dijkstra

현재 가장 싼 후보부터 탐색한다.

| 이런 조건이라면 다익스트라를 고려한다

01 최소 비용 / 최소 시간 / 최단 거리? ✓

02 그래프로 표현할 수 있는가? ✓

03 이동마다 서로 다른 비용이 있는가? ✓

04 음수 가중치가 없는가? ✓

조건으로 판단

Dijkstra

현재 가장 싼 후보부터 탐색한다.

| 이런 조건이라면 다익스트라를 고려한다

01 최소 비용 / 최소 시간 / 최단 거리? ✓

02 그래프로 표현할 수 있는가? ✓

03 이동마다 서로 다른 비용이 있는가? ✓

04 음수 가중치가 없는가? ✓

조건으로 판단

Dijkstra

현재 가장 싼 후보부터 탐색한다.

| 이런 조건이라면 다익스트라를 고려한다

01 최소 비용 / 최소 시간 / 최단 거리? ✓

02 그래프로 표현할 수 있는가? ✓

03 이동마다 서로 다른 비용이 있는가? ✓

04 음수 가중치가 없는가? ✓

조건으로 판단

Dijkstra

현재 가장 싼 후보부터 탐색한다.

| 이런 조건이라면 다익스트라를 고려한다

01 최소 비용 / 최소 시간 / 최단 거리? ✓

02 그래프로 표현할 수 있는가? ✓

03 이동마다 서로 다른 비용이 있는가? ✓

04 음수 가중치가 없는가? ✓

조건으로 판단

Dijkstra

현재 가장 싼 후보부터 탐색한다.

| 격자 문제라고 무조건 BFS가 아니다

문제의 비용 구조	알고리즘	선택 기준
단순 도달 여부	DFS / BFS	방문할 수 있는가
모든 이동 비용 동일	BFS	최소 이동 횟수
서로 다른 비용 · 음수 없음	Dijkstra	최소 누적 비용
음수 간선 존재	Bellman-Ford 등	다익스트라의 전제 불충족

중요한 것은 격자의 모양이 아니라 **간선 비용의 구조**다.

| 문제의 모양보다 비용 구조를 보자

01 격자도 그래프다.

02 이동 비용이 다르면 단순 BFS로는 부족할 수 있다.

03 다익스트라는 현재 가장 싼 후보부터 탐색한다.

04 더 싼 경로를 발견하면 dist를 갱신한다.

감사합니다.